



POLITECNICO
MILANO 1863

Streaming Data Engineering

A Tale of Four Streams

Emanuele Della Valle

prof @ Politecnico di Milano

founder @ Quantia Consulting

founder & CRO @ motus ml



POLITECNICO
MILANO 1863

The vision **Continuity matters!**

recall



POLITECNICO
MILANO 1863

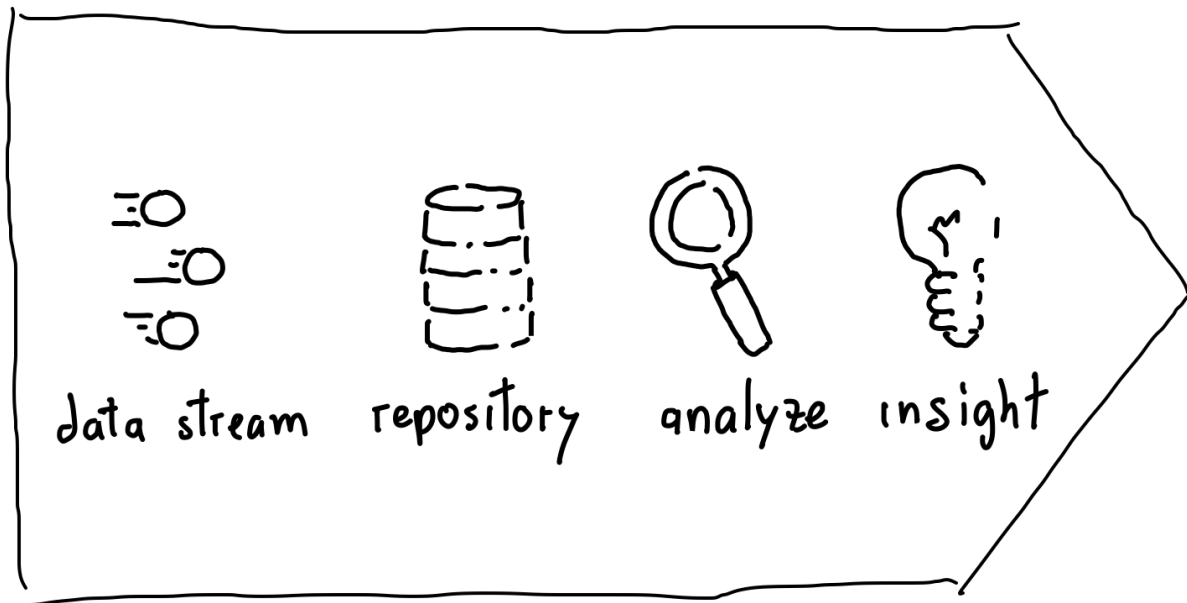
Continuity matters!



Photo by
recall

Traditional approach vs ...

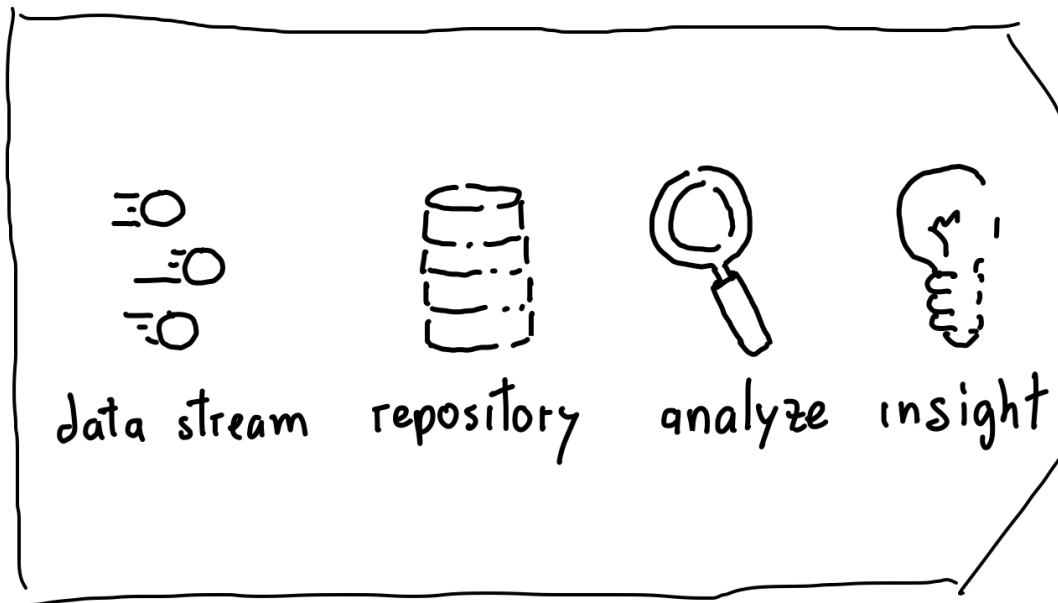
Traditional approach



Stop data to analyze

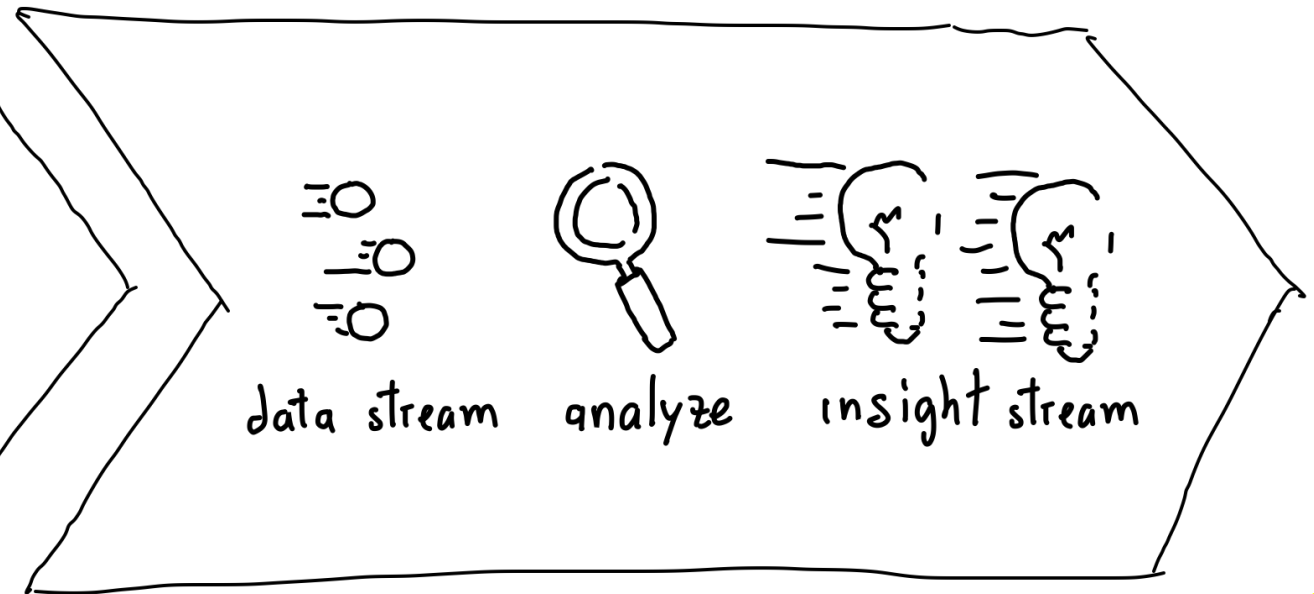
Traditional vs. Continuous approach

Traditional approach



Stop data to analyze

Continuous approach



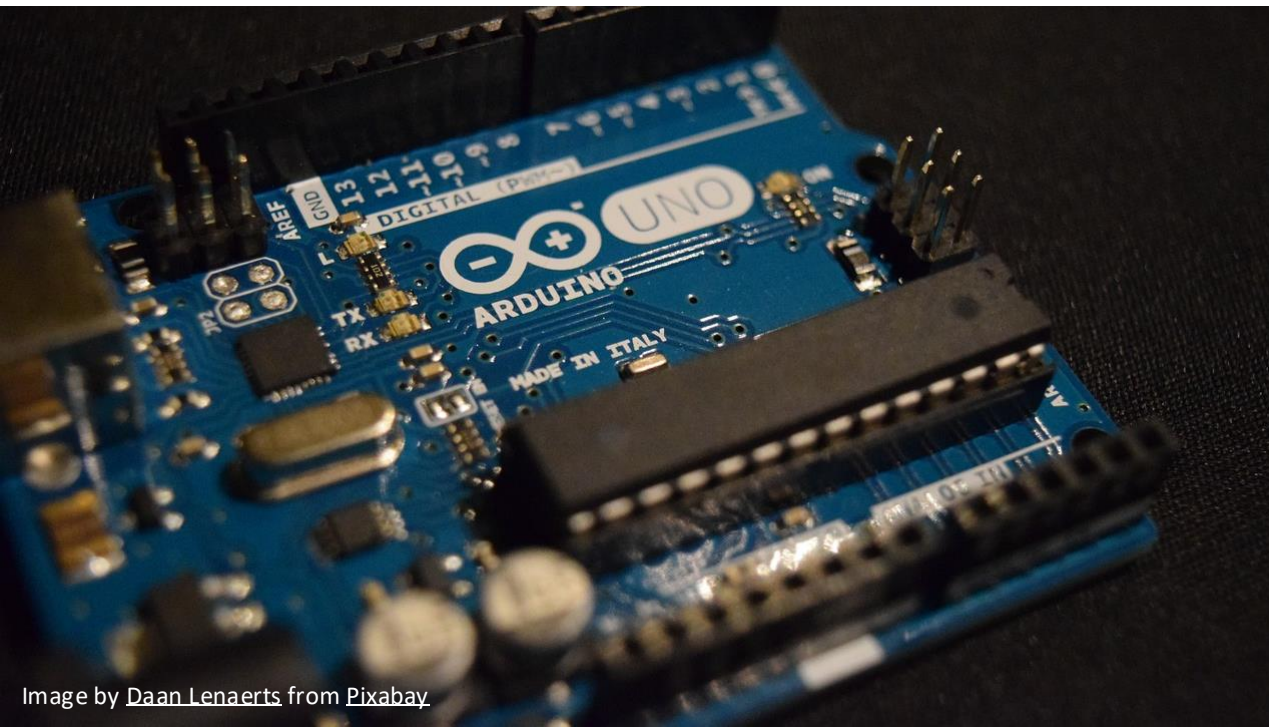
Analyze data in motion

recall



1st key concept
(Data) Streams
take many forms

**Myriads of tiny flows
that you can collect**



**e.g., from physical or
software sensors**



**Continuous massive flows
than you cannot stop**

e.g., telcos' and
utilities' **monitoring**



e.g., physical or cyber
alarms



Image by [Dieter zabel](#) from [Pixabay](#)



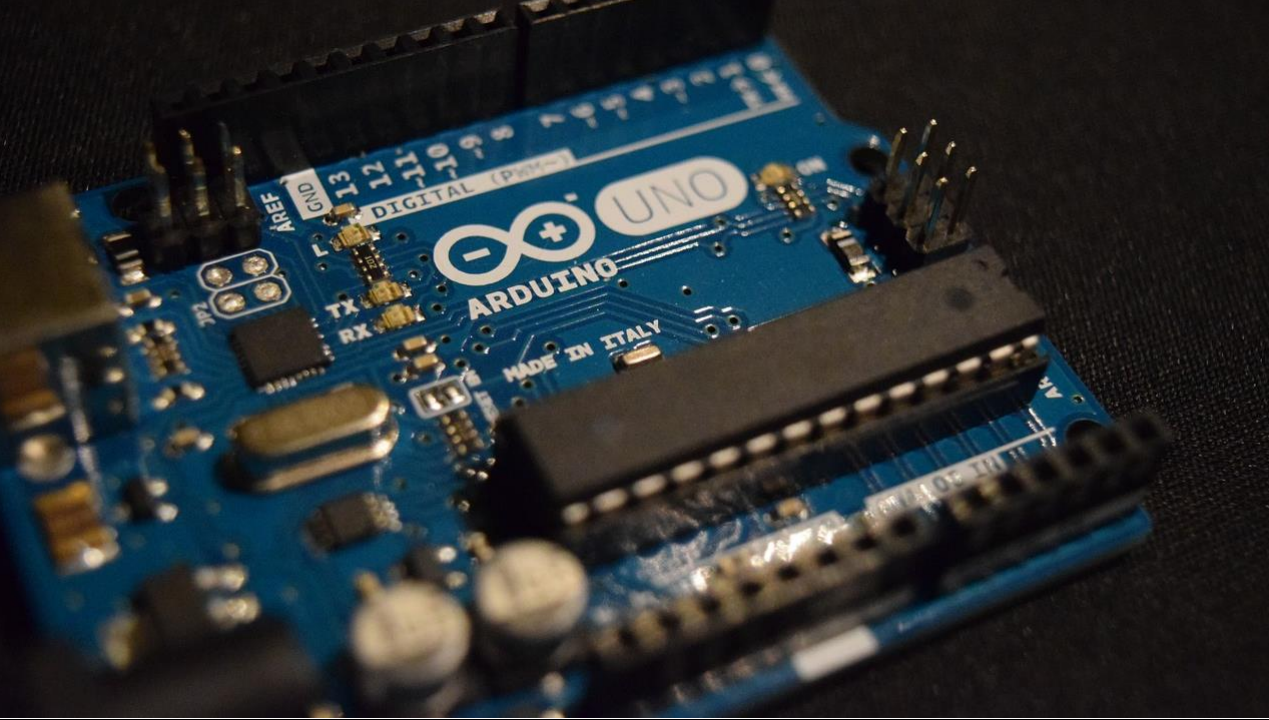
**Continuous numerous flows
that can turn into a torrent**

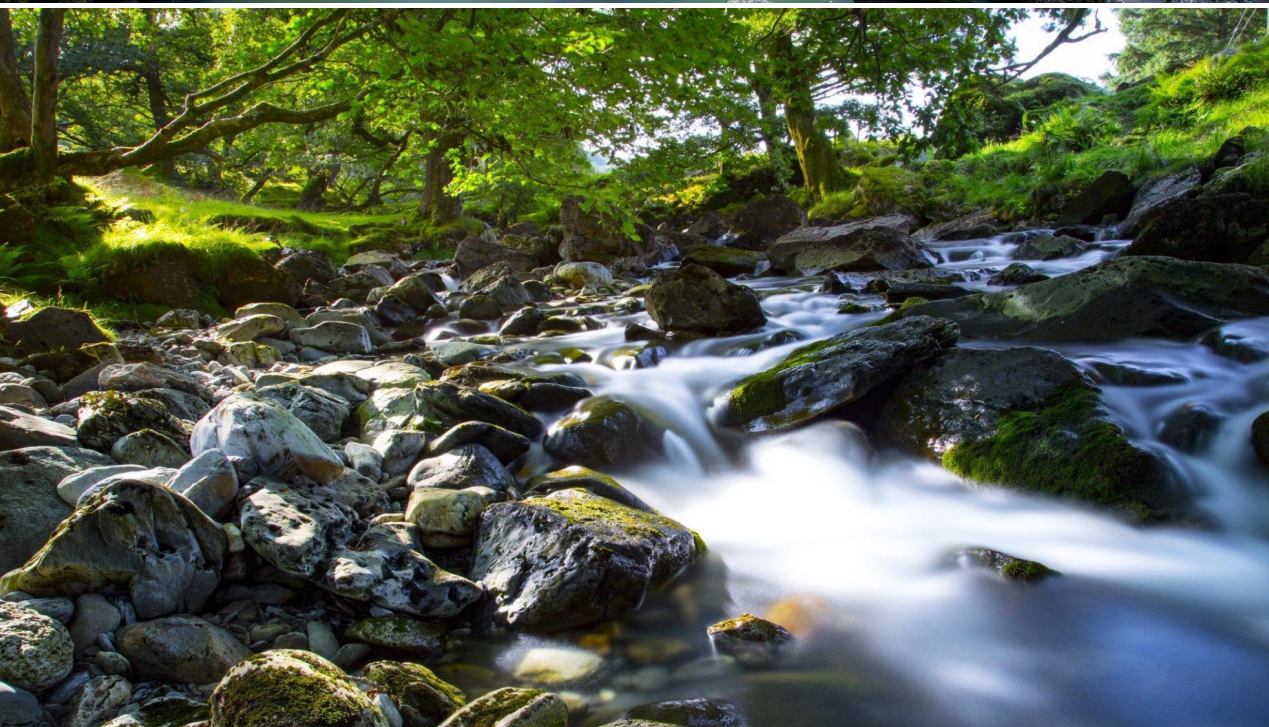


e.g., to physical or software
actuatorsMyriads

**Myriads of flows of any
size and speed forming
an immense delta**







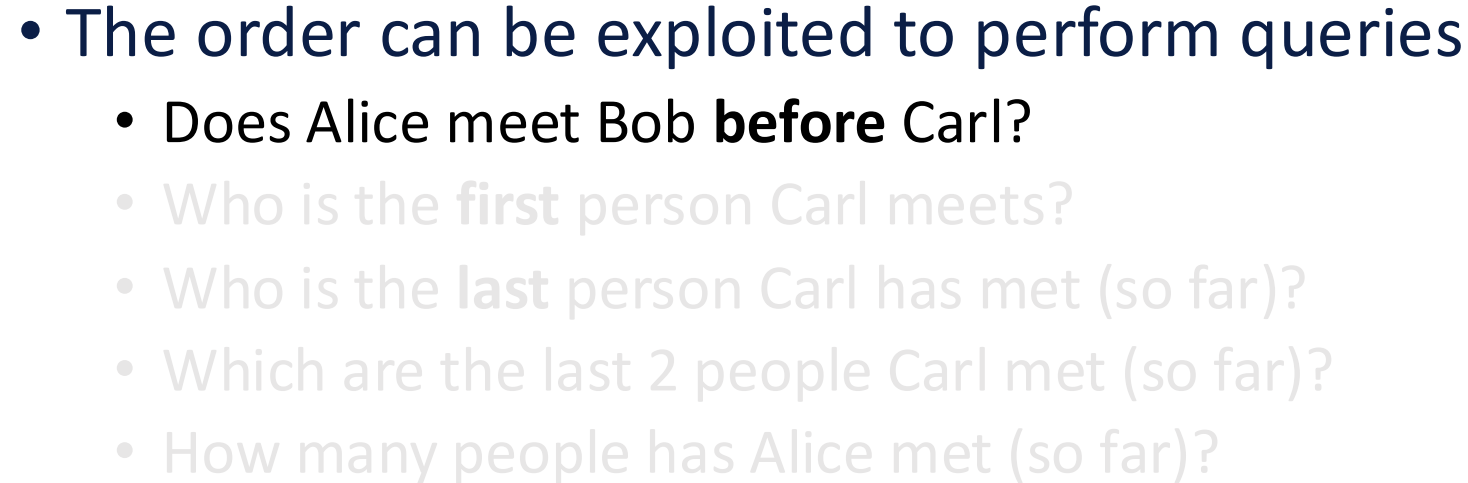


POLITECNICO
MILANO 1863

2nd key concept

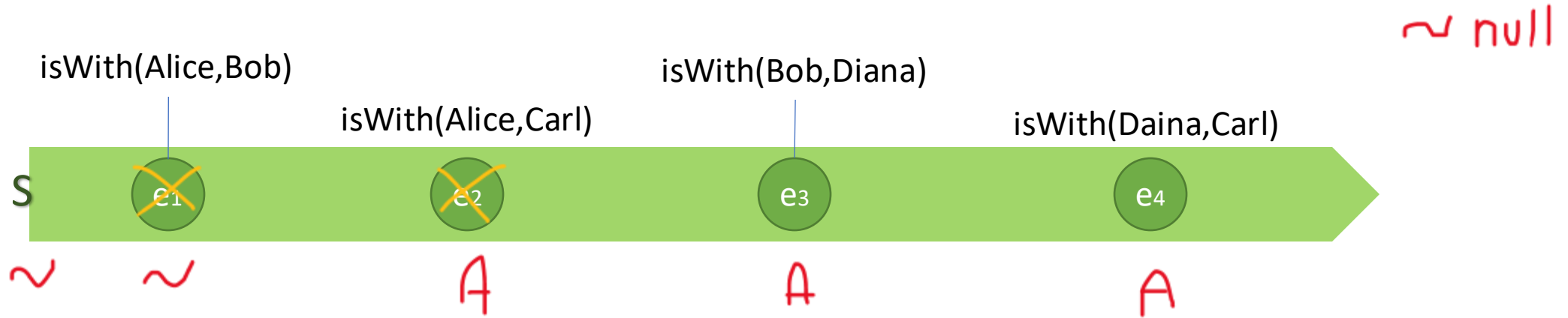
Time models matter!

0 no
1 yes
~ null



@manudellavalle - <http://emanueledellavalle.org>

Stream-only Time Model

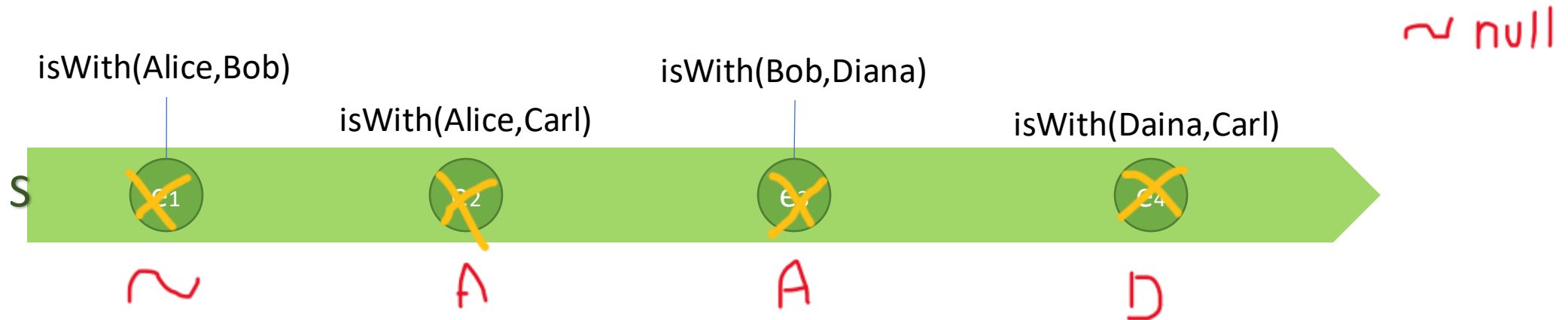


- The order can be exploited to perform queries

- Does Alice meet Bob **before** Carl?
- Who is the **first** person Carl meets?
- Who is the **last** person Carl has met (so far)?
- Which are the last 2 people Carl met (so far)?
- How many people has Alice met (so far)?

NOTE: the answer changes over time and once used the events can be forgotten!

Stream-only Time Model

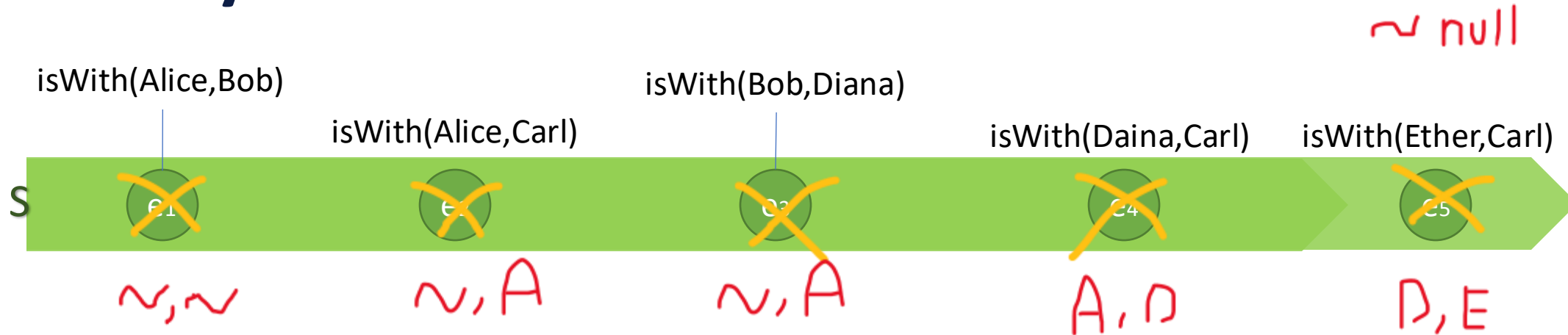


- The order can be exploited to perform queries

- Does Alice meet Bob **before** Carl?
- Who is the **first** person Carl meets?
- Who is the **last** person Carl has met (so far)?
- Which are the last 2 people Carl met (so far)?
- How many people has Alice met (so far)?

NOTE: the answer changes over time and once used the events can be forgotten!

Stream-only Time Model

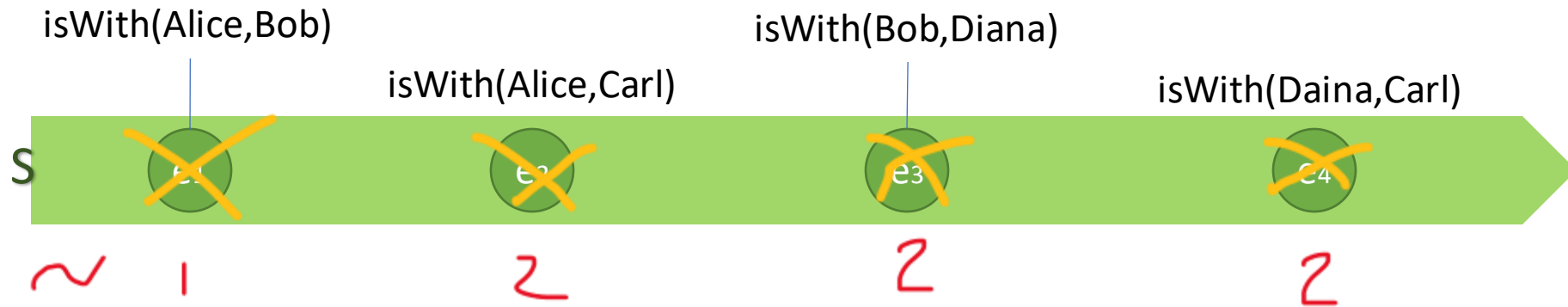


- The order can be exploited to perform queries

- Does Alice meet Bob **before** Carl?
- Who is the **first** person Carl meets?
- Who is the **last** person Carl has met (so far)?
- Which are the **last 2** people Carl met (so far)?
- How many people has Alice met (so far)?

NOTE: the answer changes over time and once used the events can be forgotten!

Stream-only Time Model



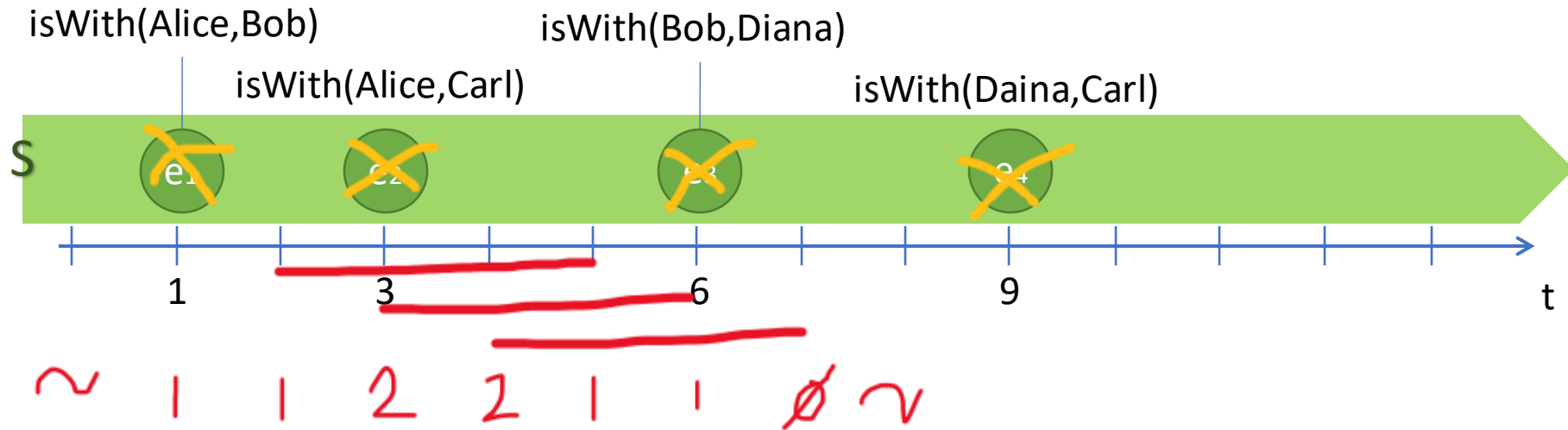
- The order can be exploited to perform queries

- Does Alice meet Bob **before** Carl?
- Who is the **first** person Carl meets?
- Who is the **last** person Carl has met (so far)?
- Which are the last 2 people Carl met (so far)?
- **How many** people has Alice met (so far)?

NOTE: the answer changes over time and once used the events can be forgotten!

Absolute Time Model

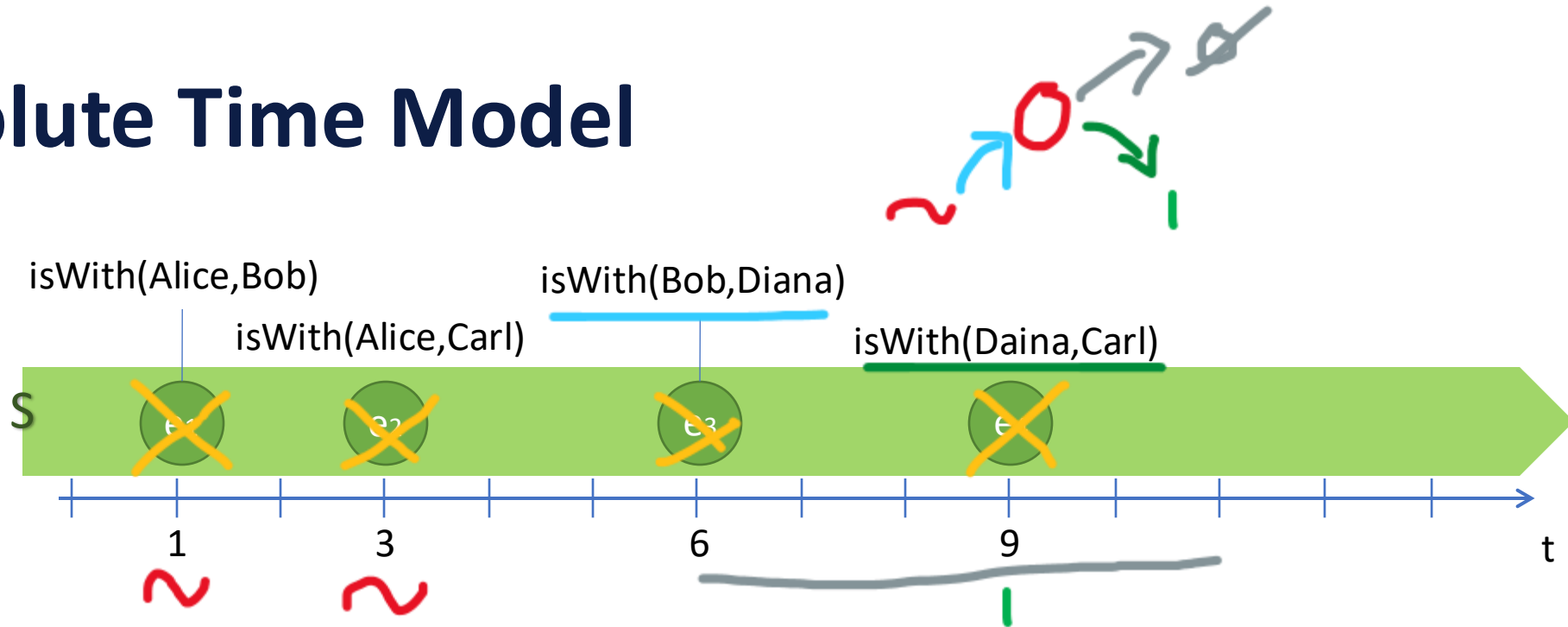
~ null



- We can ask the queries in the previous slide
- We can start to compose queries taking into account the time
 - How many people has Alice met **in the last 4 m?**
 - Does (every time) Diana meet Bob and then Carl **within 5m?**

NOTE: the answer changes over time and once used the events can be forgotten!

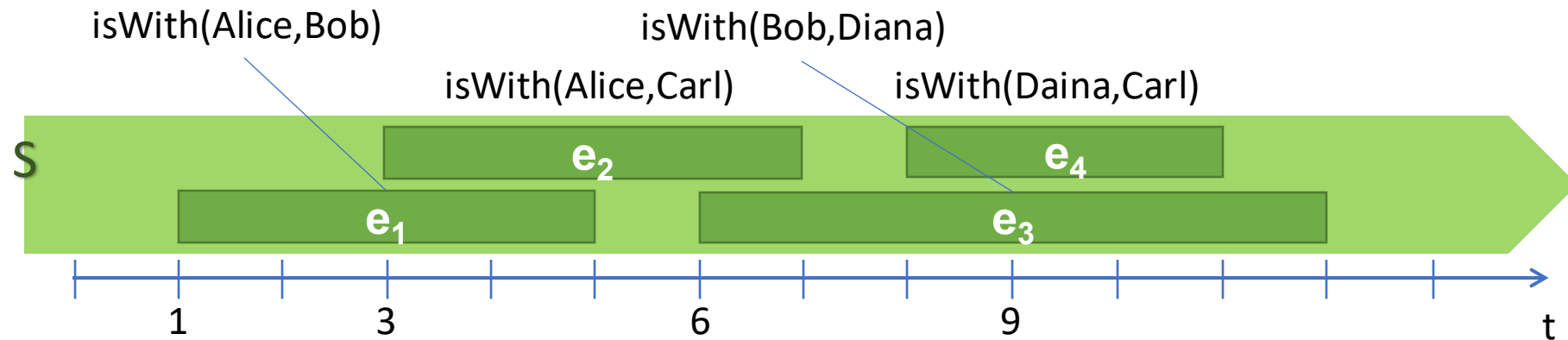
Absolute Time Model



- We can ask the queries in the previous slide
- We can start to compose queries taking into account the time
 - How many people has Alice met in the last 5m?
 - Does (every time) Diana meet Bob and then Carl within 5m?

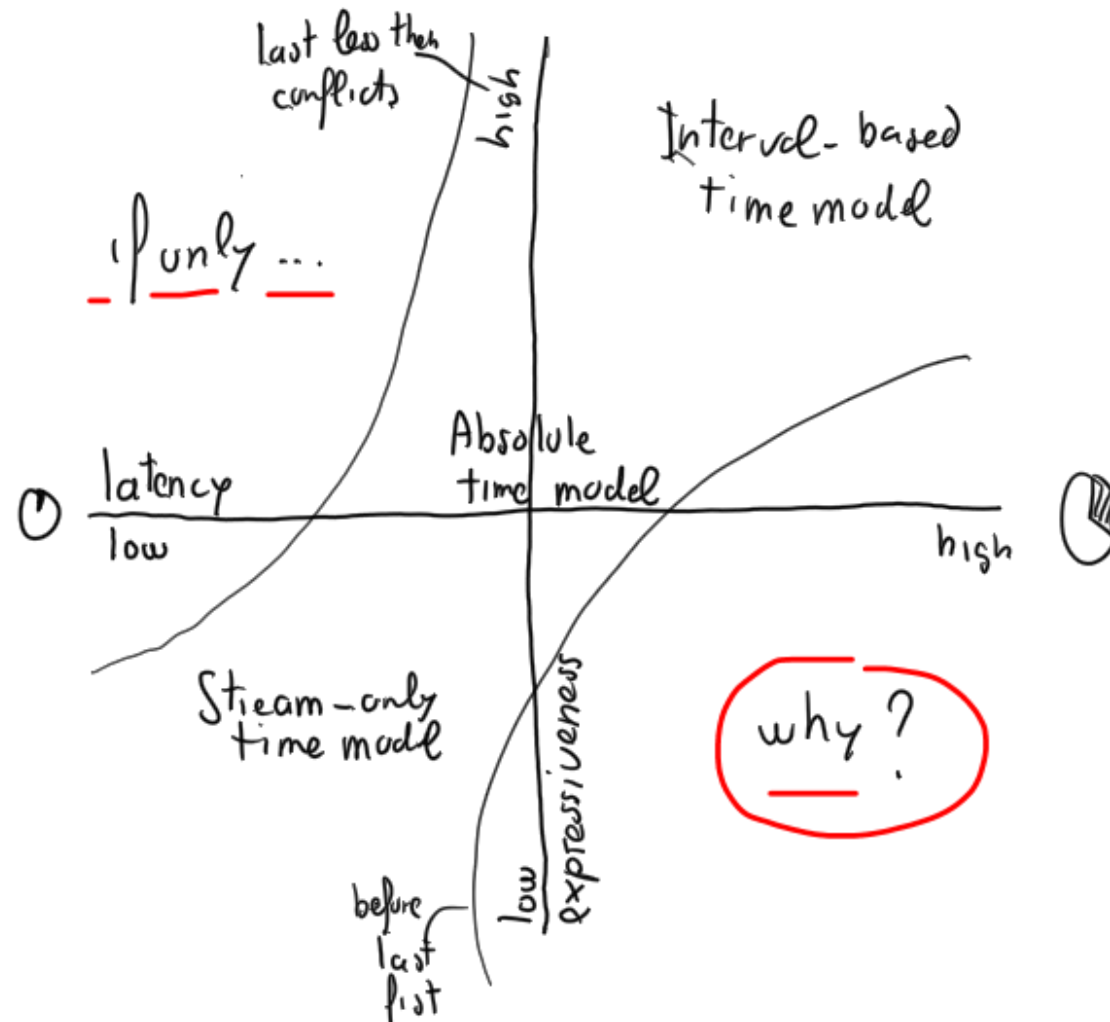
NOTE: the answer changes over time and once used the events can be forgotten!

Interval-base Time Model



- We can ask the queries in the previous two slides
- It is possible to write even more complex queries:
 - Which are the meetings the **last less than 5m**?
 - Which are the meetings with **conflicts**?

A first trade-off: latency vs. expressiveness





3rd key concept
No solution
to tame all streams

A long-exposure photograph of a waterfall cascading over mossy rocks into a pool. The water is blurred, creating a soft, white flow. The surrounding area is lush with green moss and vegetation. The text is overlaid on the right side of the image.

Taming
*Myriads of tiny flows
that you can collect*
with
Event-based Systems

Image by [Pexels](#) from [Pixabay](#)

A definition

- An Event-Based System is a software architecture paradigm in which data flows and operations are driven by events rather than following a predefined control flow (i.e., they are decoupled).
- In other words, in an event-based system, actions are executed in response to specific events that occur rather than as part of a predetermined logical sequence.



On Events and decoupling

Events

an immutable and append-only stream of “business facts”

Decoupling

means decentralizing the freedom to act, adapt, and change

Types of EBS: content- vs. topic-based

Content-based systems

- filter and route messages based on the actual content within the messages.
- Subscribers specify their interests through content attributes, and they only receive messages that match these specific criteria
- Mainly an academic effort



Types of EBS: content- vs. topic-based

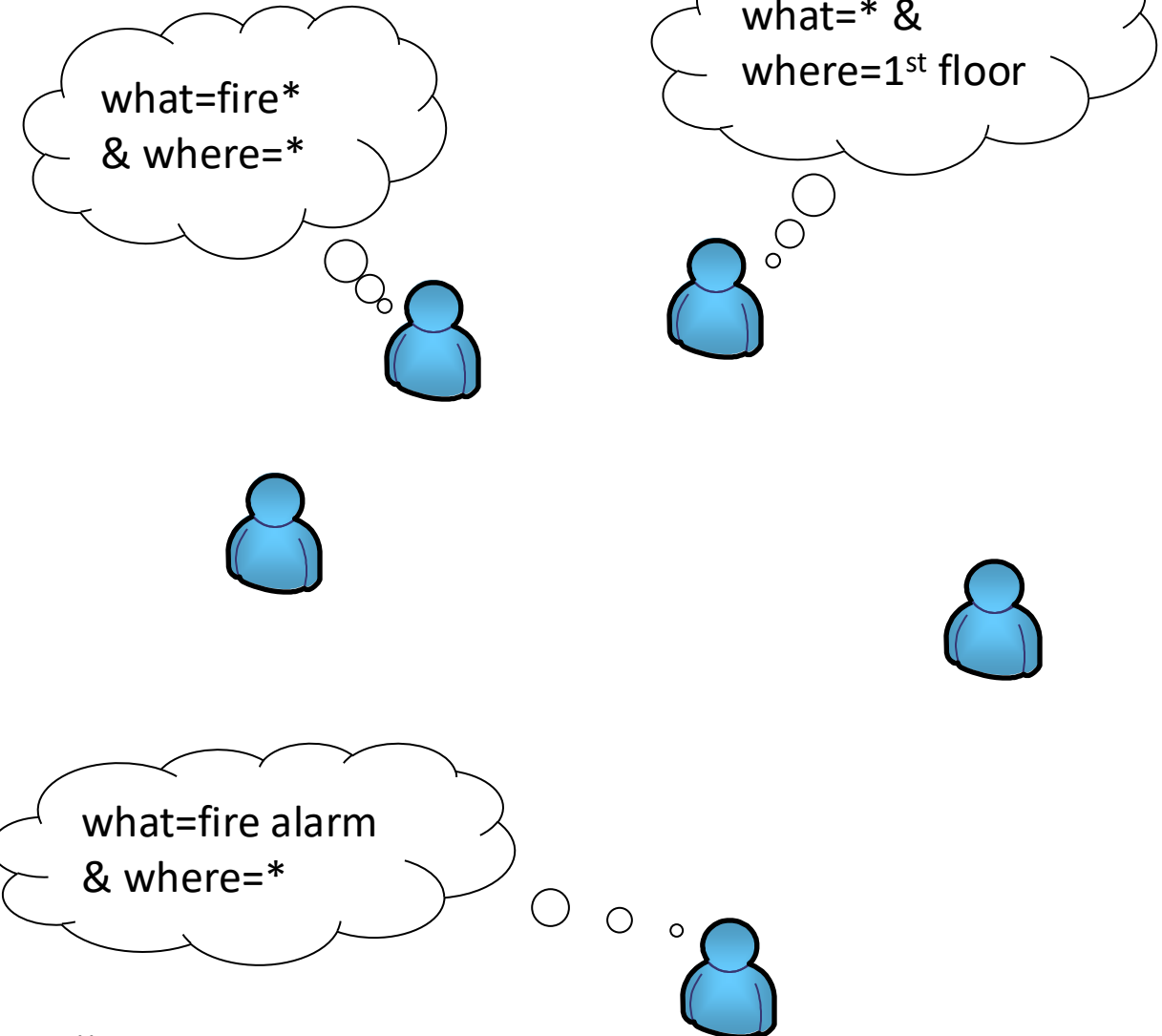


Topic-based systems

- organize messages by predefined topics
- Publishers send messages to a specific topic
- Subscribers receive all messages sent to the topics they have subscribed to
- Widely available for industrial applications

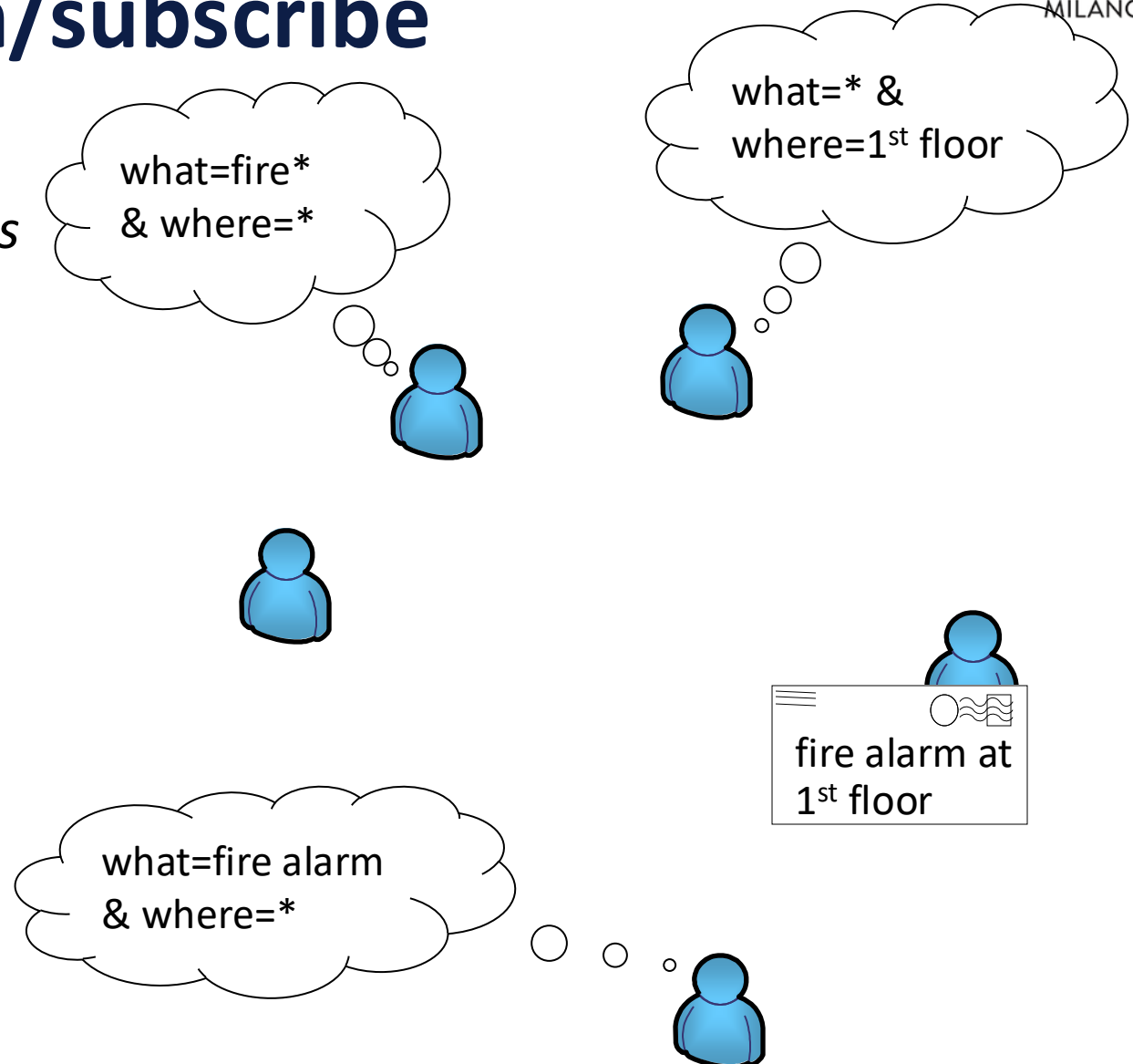
Content based Publish/subscribe

- Components
 - collaborate by exchanging *events*
 - *publish events they observe*
 - *subscribe* to the events they are interested in
- Communication is:
 - Purely message based
 - Asynchronous
 - Multicast
 - Implicit
 - Anonymous



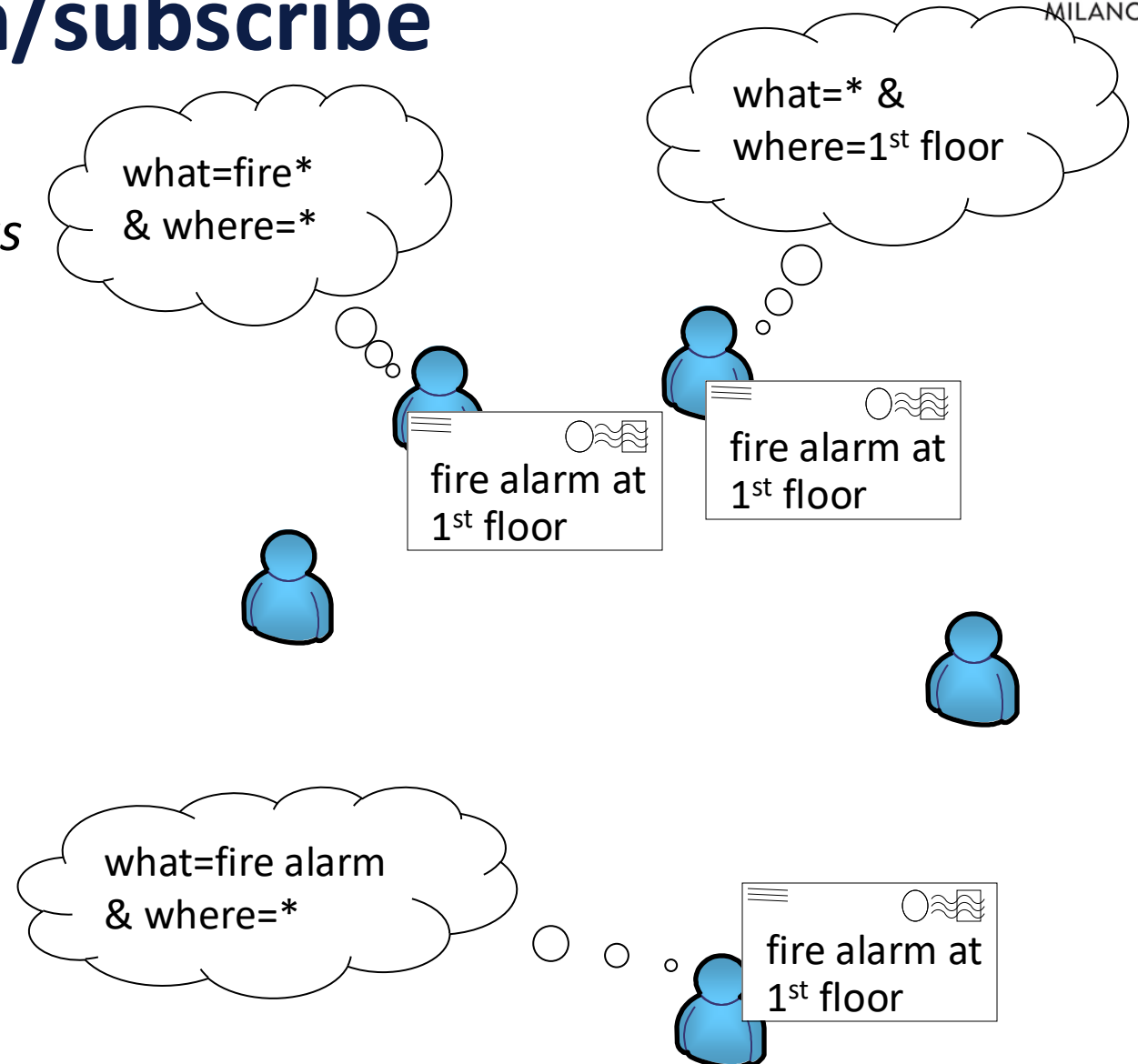
Content based Publish/subscribe

- Components
 - collaborate by exchanging *events*
 - *publish events they observe*
 - *subscribe* to the events they are interested in
- Communication is:
 - Purely message based
 - Asynchronous
 - Multicast
 - Implicit
 - Anonymous



Content based Publish/subscribe

- Components
 - collaborate by exchanging *events*
 - *publish events they observe*
 - *subscribe* to the events they are interested in
- Communication is:
 - Purely message based
 - Asynchronous
 - Multicast
 - Implicit
 - Anonymous



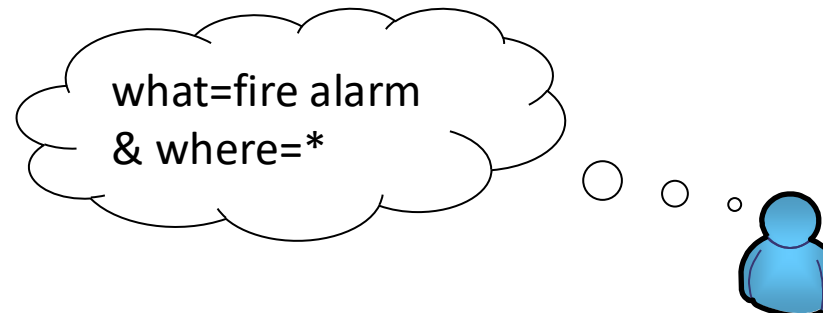
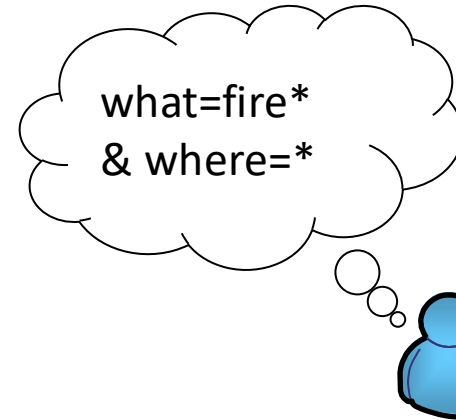
Content based Publish/subscribe

- Components

- collaborate by exchanging *events*
- *publish events they observe*
- *subscribe* to the events they are interested in

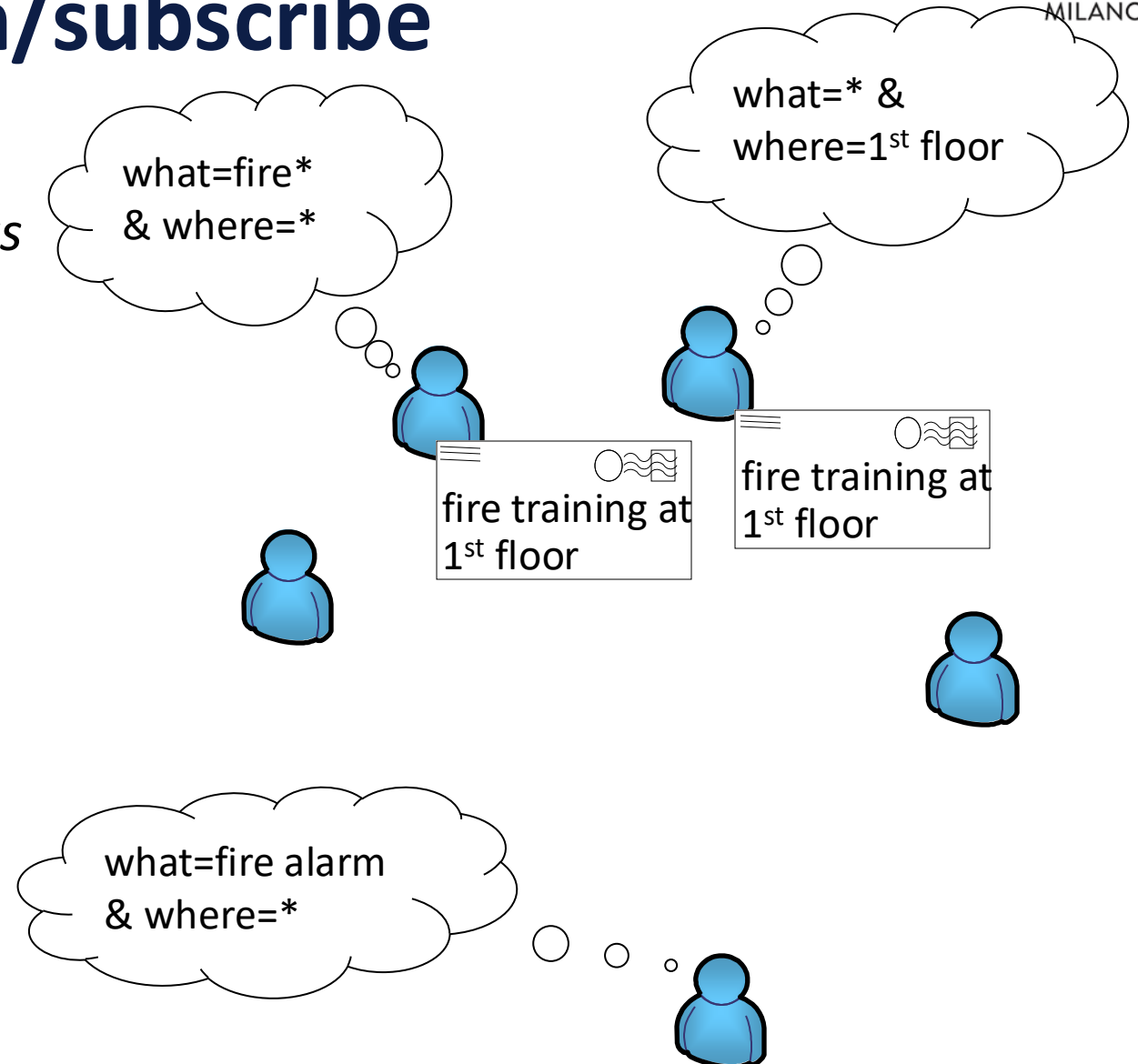
- Communication is:

- Purely message based
- Asynchronous
- Multicast
- Implicit
- Anonymous



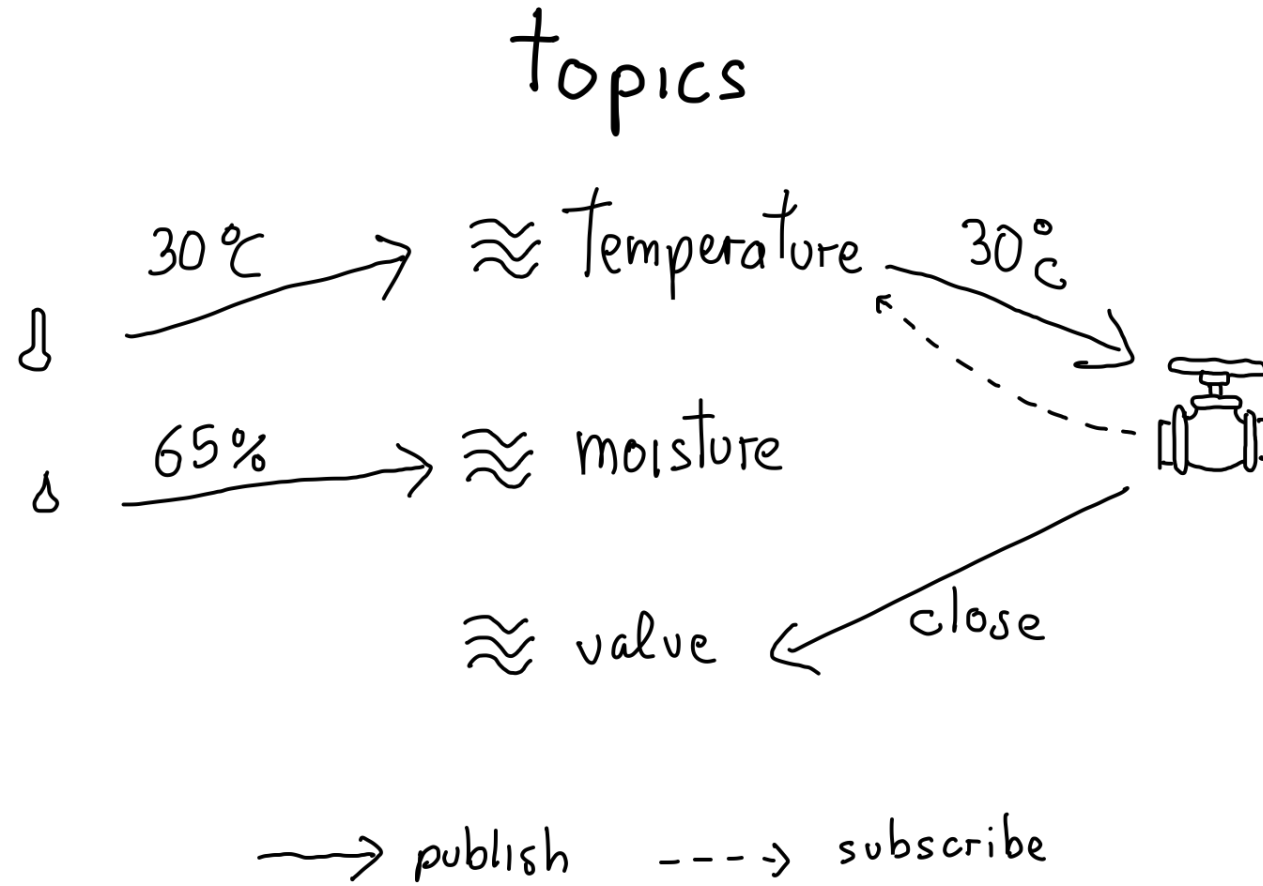
Content based Publish/subscribe

- Components
 - collaborate by exchanging *events*
 - *publish events they observe*
 - *subscribe* to the events they are interested in
- Communication is:
 - Purely message based
 - Asynchronous
 - Multicast
 - Implicit
 - Anonymous



Topic-based Publish/subscribe

e.g., MQTT & Apache Kafka

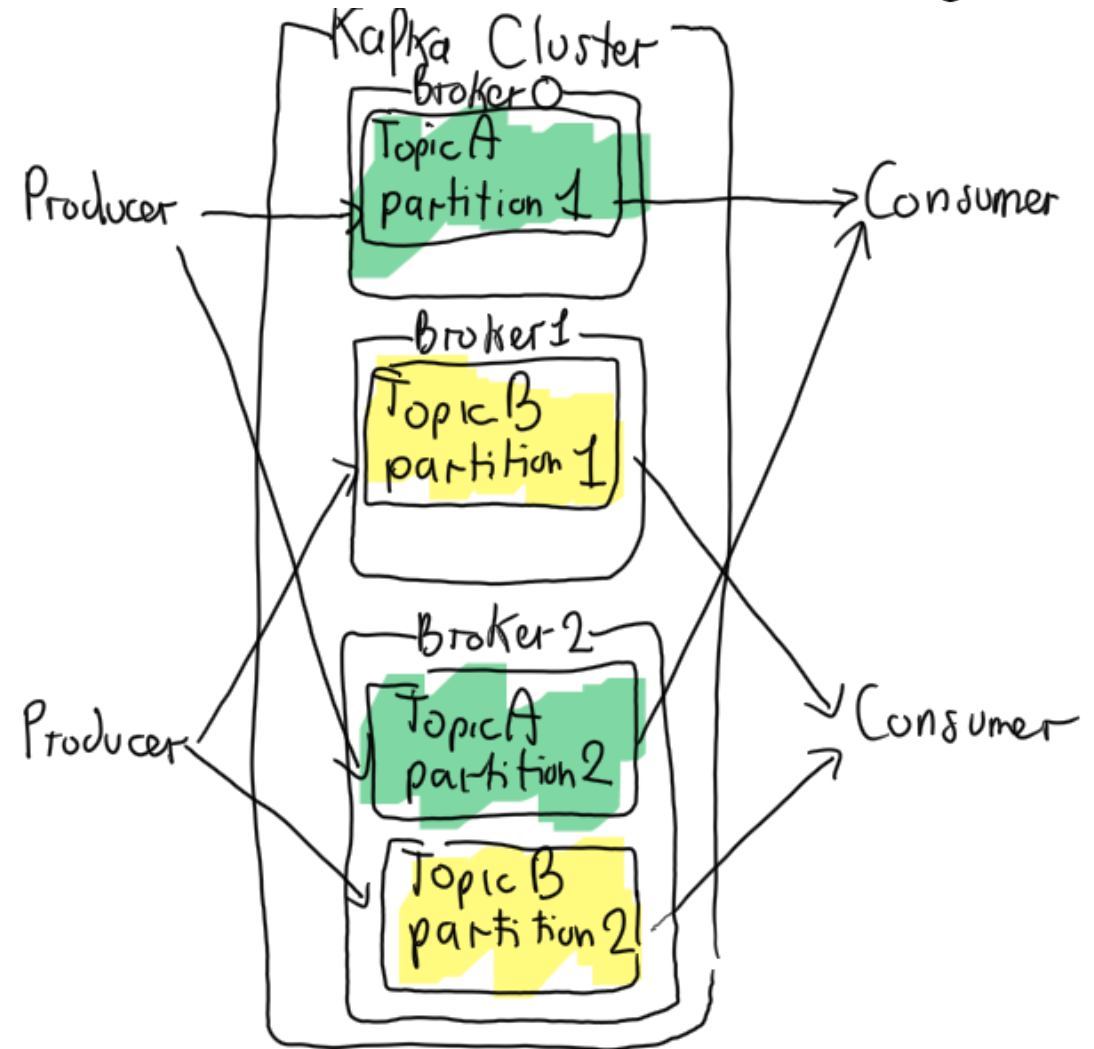




Kafka: “the” event-based system



- **Topics** are **partitioned** across Kafka brokers
- **Producers** **shard** messages over the partitions of a specific topic
- Different **Consumers** can **read in parallel** messages from different partitions of the same topic

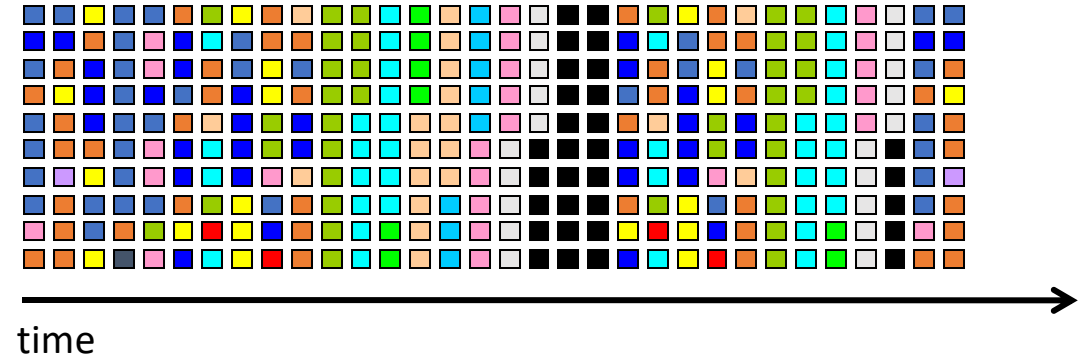




Taming
*Continuous massive flows
than you cannot stop
with*
Data Stream Mng. Systems

Data Stream Management Systems (DSMS)

- Data streams are **(unbounded)** sequences of time-varying data elements
- Represent:
 - an (almost) “continuous” flow of information
 - with the recent information being more relevant as it describes the current state of a dynamic system



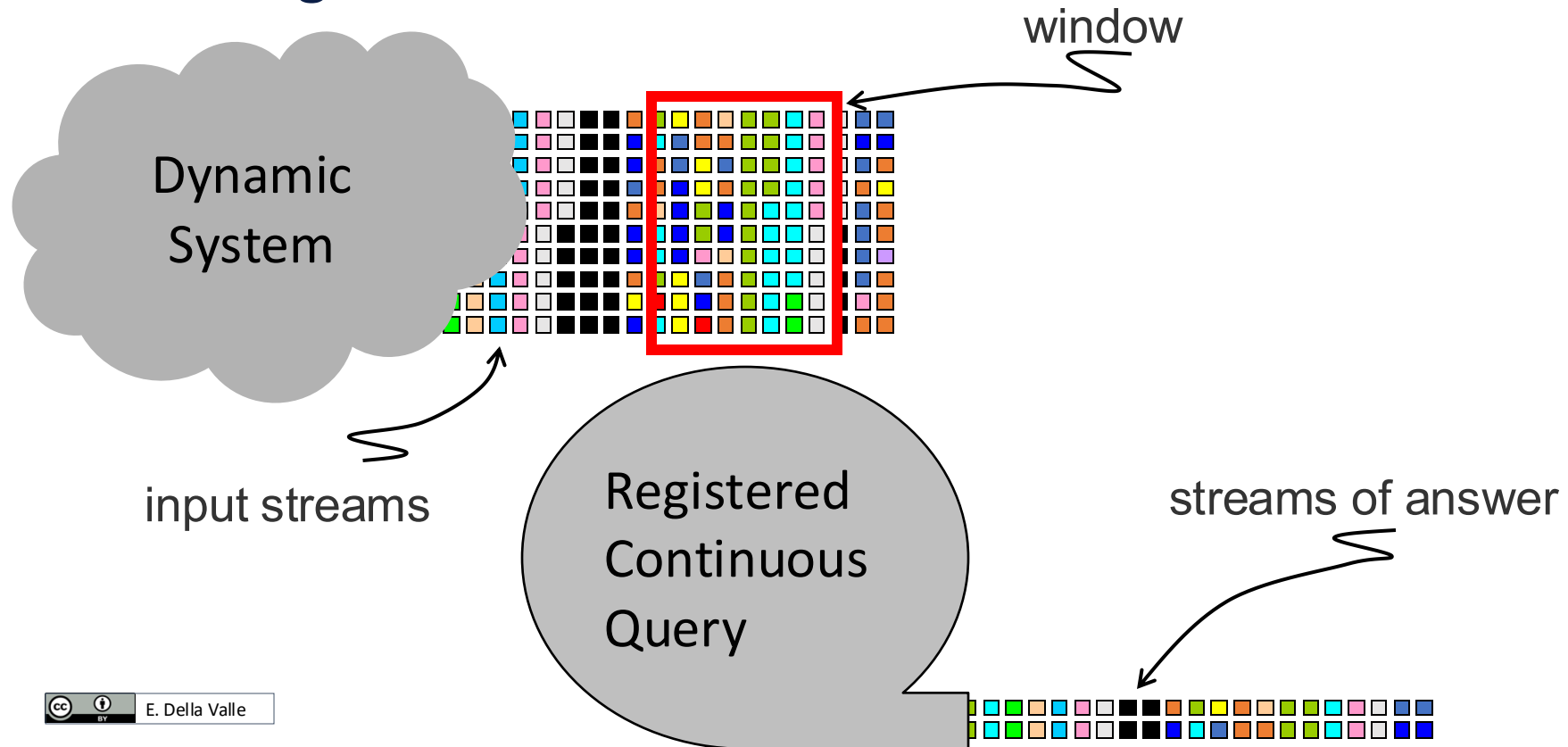
Data Stream Management Systems (DSMS)

- The nature of streams requires a paradigmatic change*
 - **from persistent data**
 - one time semantics
 - **to transient data**
 - continuous semantics

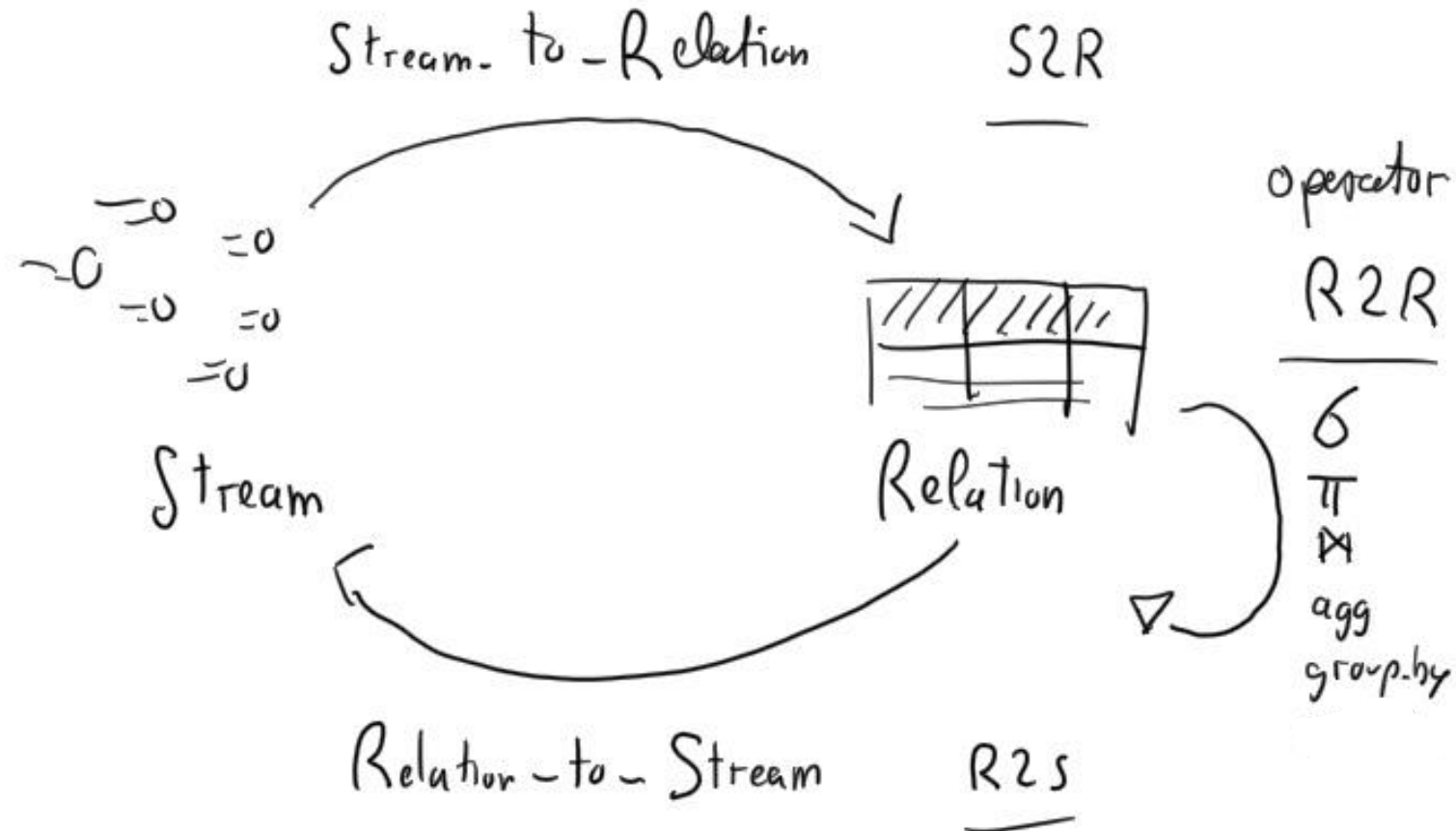
* This paradigmatic change first arose in DB community in **the late '90s**

Continuous Semantics

- Continuous **queries registered** over streams that are observed through **windows**



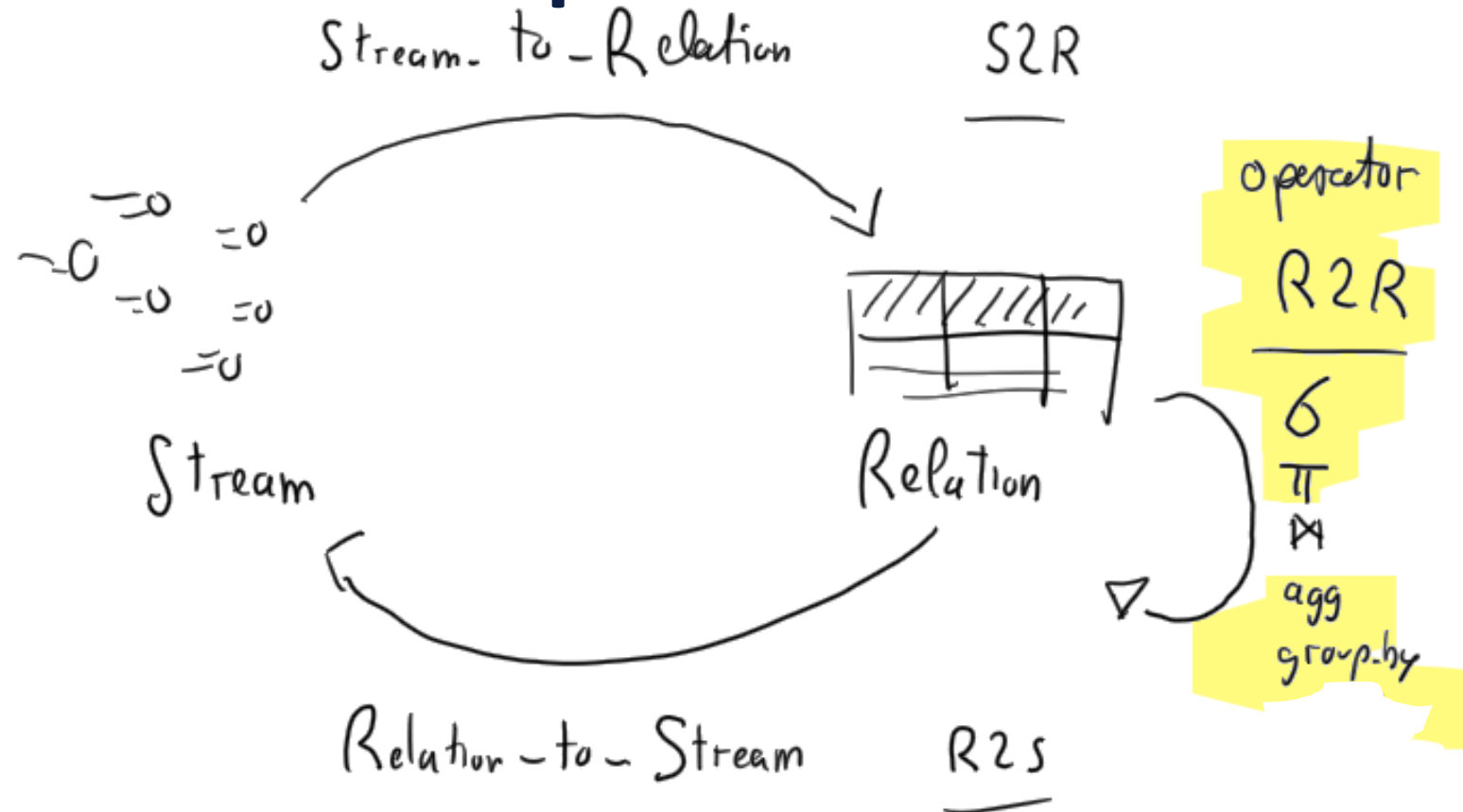
DSMS Semantics



Arasu, A., Babu, S. and Widom, J., 2006. **The CQL continuous query language: semantic foundations and query execution.** *The VLDB Journal*, 15(2), pp.121-142.

DSMS Semantics

Relation to Relation operators



Arasu, A., Babu, S. and Widom, J., 2006. **The CQL continuous query language: semantic foundations and query execution.** *The VLDB Journal*, 15(2), pp.121-142.

Modern DSMS

Logic View
Unbounded Table

K	V
♥	2
♥	7
♦	4
♥	3

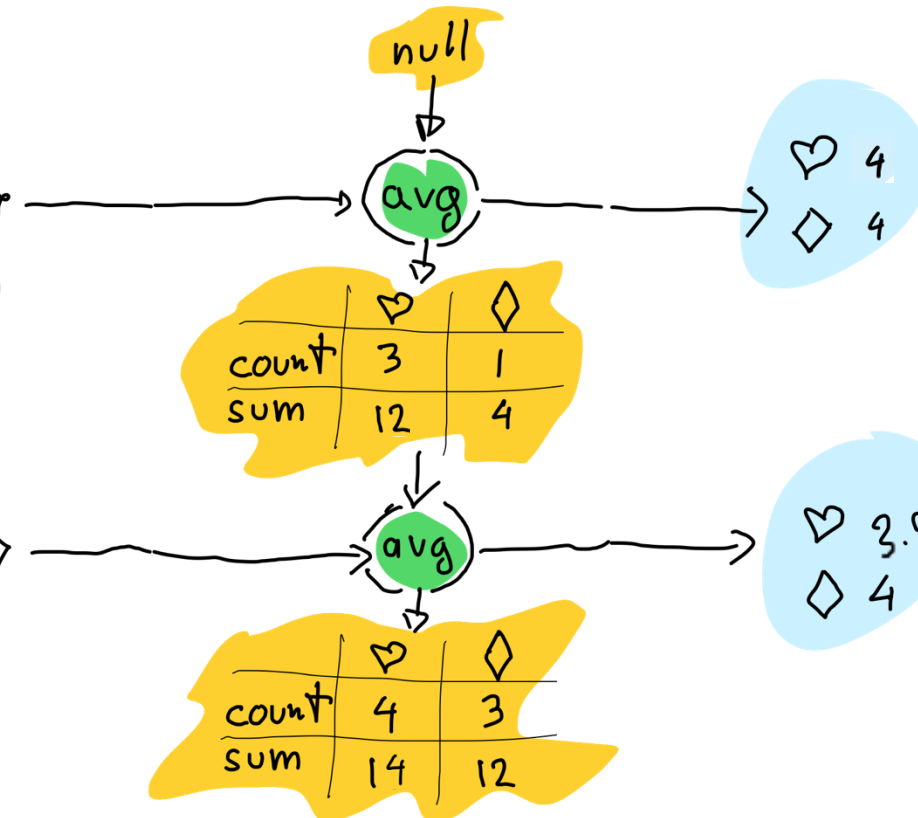
+

♦	2
♦	6
♥	2

+

⋮

Physical View
Incremental Evaluation



state

operator

results





Doubt

Is it possible to perform all tasks incrementally?
by only looking at the data once
and by using a finite, small amount of memory



Is it possible to perform all tasks incrementally?

Possible and exact	Possible but approximated	Impossible
Filters Mapping Grouping Count, sum, max, min, avg Variance Standard-deviation	Median Percentile Mode & Entropy Skew & Kurt Covariance Pearson Corr.	Total order Distinct
Exact results in a single pass using a finite state	Exact results require multiple passes or an unbounded state	Same as previous, but approximation is unacceptable

Certainly, incremental operators

- Filters ... more generally mapping¹
- Grouping
- Count, sum, max, min, avg
- Variance and standard-deviation²

¹ an operation that associates each element of a given set (the domain) with one or more elements of a second set (the range).

² it is not possible with the usual two-pass algorithm, see next slide

Welford's Algorithm for Incremental Variance Calculation

- Initialize:
 - the number of points processed so far: $n = 0$,
 - the current mean: $mean = 0$
 - the sum of squares of differences from the current mean: $M2 = 0$
- For each new data point x :
 - Increment n : $n = n + 1$
 - Compute $delta$: $delta = x - mean$
 - Update $mean$: $mean = mean + delta / n$
 - Update $M2$: $M2 = M2 + delta * (x - mean)$
 - Calculate $variance$: $variance = M2 / (n - 1)$
 - Calculate $Standard Deviation$: $std_dev = sqrt(variance)$

What about these?

- Median
- Percentile
- **No** because, in the **worst case**, you need an **(infinite) state** (i.e., as large as the stream that is unbounded)
- **Almost** in an **approximated** way
 - Raj Jain and Imrich Chlamtac. [The P2 algorithm for dynamic calculation of quantiles and histograms without storing observations](#). Commun. ACM, 28(10):1076–1085, oct 1985.
 - Michael Greenwald and Sanjeev Khanna. [Space-efficient online computation of quantile summaries](#). ACM SIGMOD Record, 30(2):58–66, 2001.

What about these?

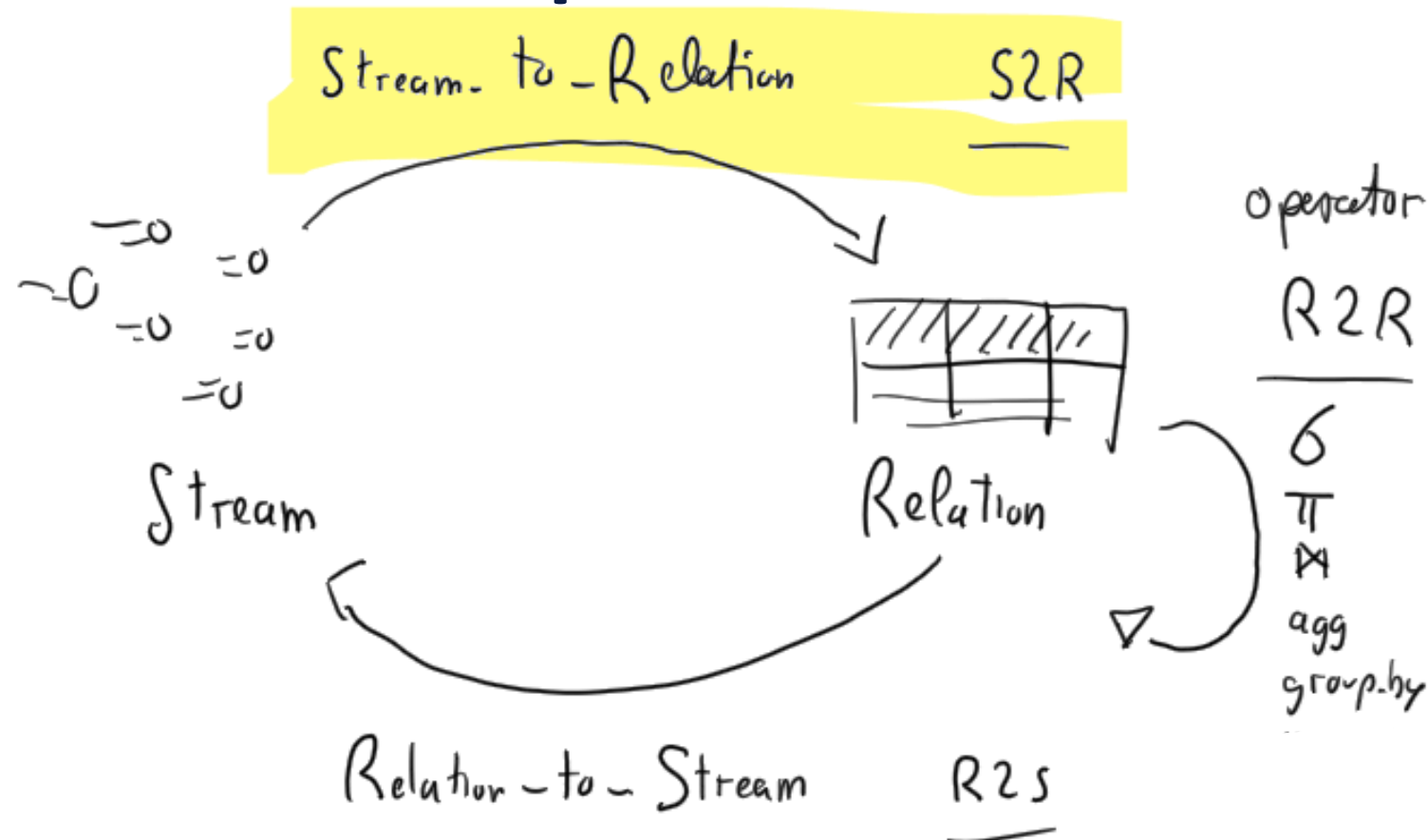
- Mode & Entropy
 - Skew & Kurt
 - Covariance & Pearson Corr.
-
- **No** because you need **two passes** over the data
-
- **Almost, approximated** streaming algorithms exist
 - E.g., Cormode, G. and Muthukrishnan, S., 2005. [An improved data stream summary: the count-min sketch and its applications.](#) *Journal of Algorithms*, 55(1), pp.58-75.
 - But they are still actively investigated

What about these?

- Total order
- Distinct
- **No** because, in the **worst case**, you need an **(infinite) state** (i.e., as large as the stream that is unbounded)
- **But**, possible for a **window**, i.e., the Stream to Relation operators

DSMS Semantics

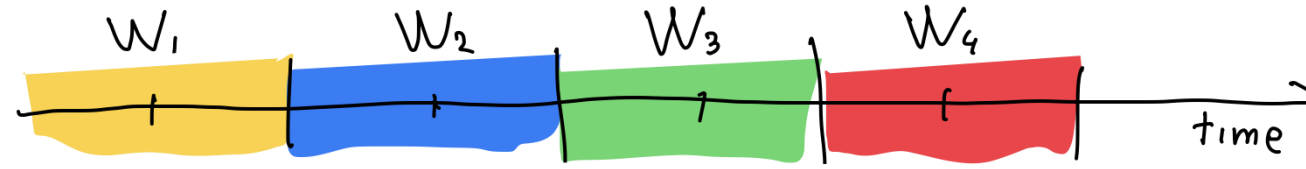
Relation to Relation operators



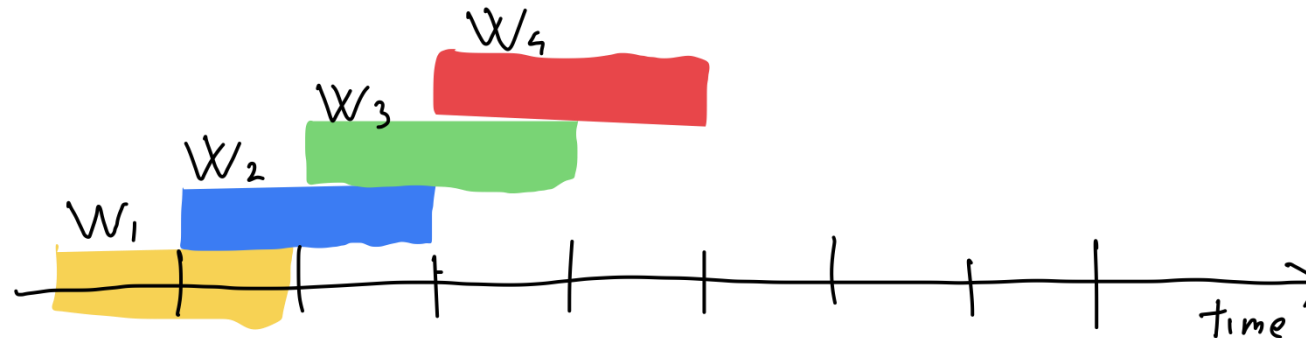
Arasu, A., Babu, S. and Widom, J., 2006. **The CQL continuous query language: semantic foundations and query execution.** *The VLDB Journal*, 15(2), pp.121-142.

(The most important) Windows

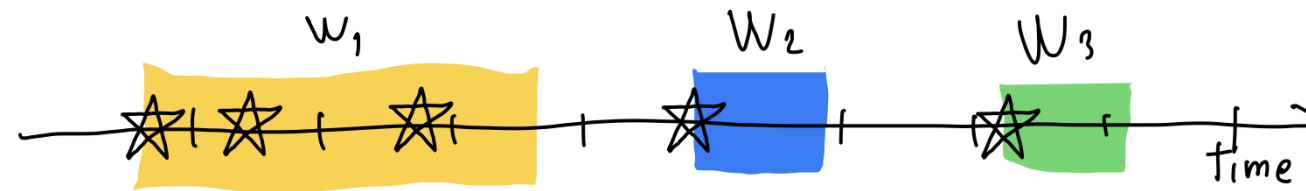
Tumbling
size



Hopping
size & stride



Session
gap



(The most important) Windows (cont.)

- Note that the time model and the time unit are voluntarily unspecified.
- If we consider a stream-only time model, we define **physical windows**
 - E.g., tumbling window: every 10 events
 - E.g., hopping window: the last 10 events every 5 events
 - i.e., session windows cannot be specified
- If we consider an absolute time model, we define **logical windows**
 - E.g., tumbling window: every 10 minutes
 - E.g., hopping window: the last 10 minutes every 5 minutes
 - i.e., session window: events closer than 5 minutes (or inactivity gap larger than 5 minutes)

SQL Extensions, e.g. EPL by EsperTech

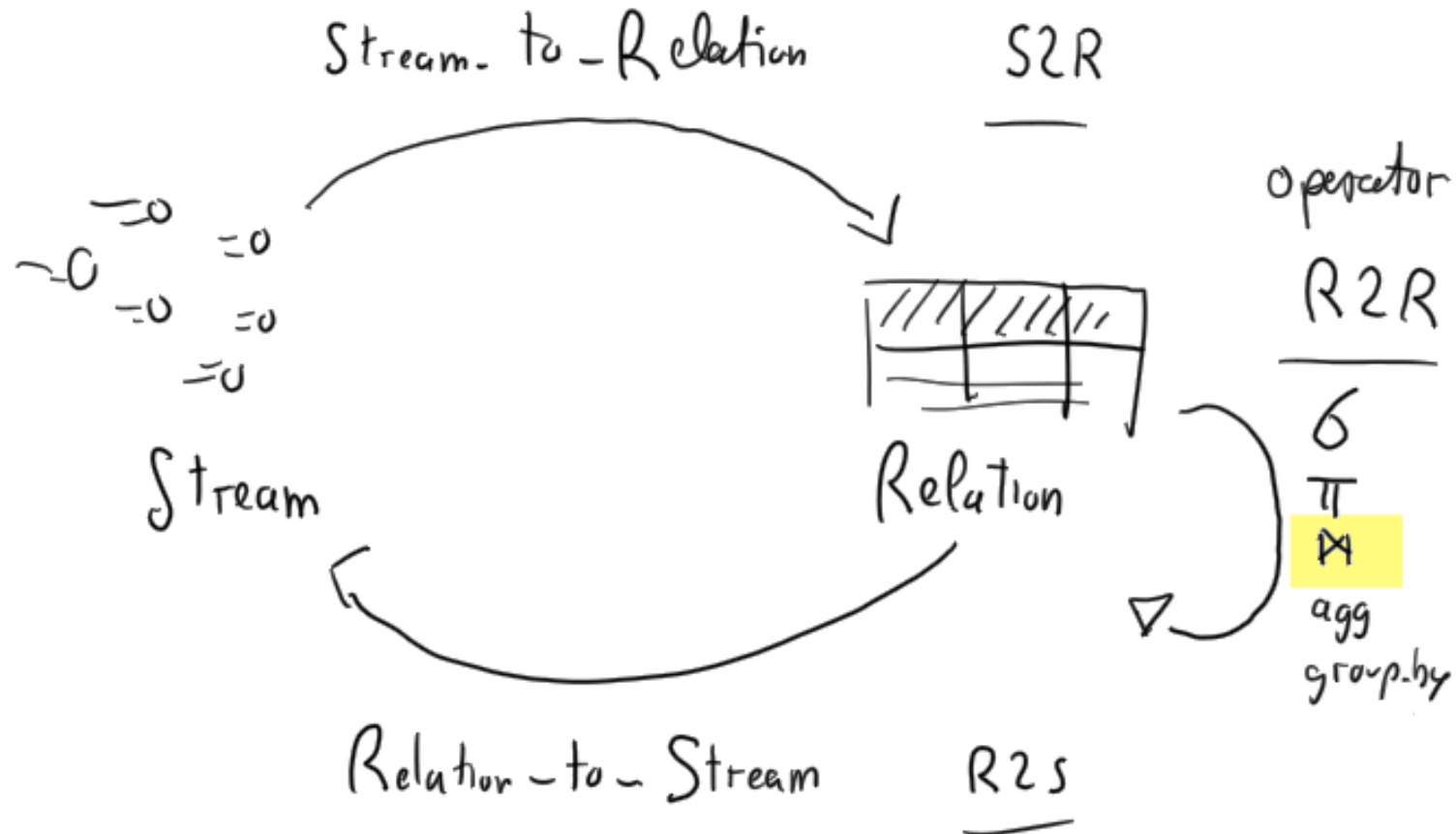
Windowed aggregation

Can you show me the average temperature for each sensor over the last 10 seconds, and update it every 5 seconds?

```
select sensor, avg(temperature)
from TemperatureStream.win:time(10 seconds)
group by sensor
output last every 5 seconds;
```

DSMS Semantics

Joins



Arasu, A., Babu, S. and Widom, J., 2006. **The CQL continuous query language: semantic foundations and query execution.** *The VLDB Journal*, 15(2), pp.121-142.

Streaming joins

- A streaming join is an operation that **combines** two or more **data streams** based on a common key or specific condition
- In traditional databases, combining rows from different tables is a basic operation, but **in stream processing is hard** because the data is not static but in constant motion
- Streams are data that flows over time. Therefore, **joins consider the temporal synchronization of the data via time windows** to limit the join to events that occur within a certain time interval

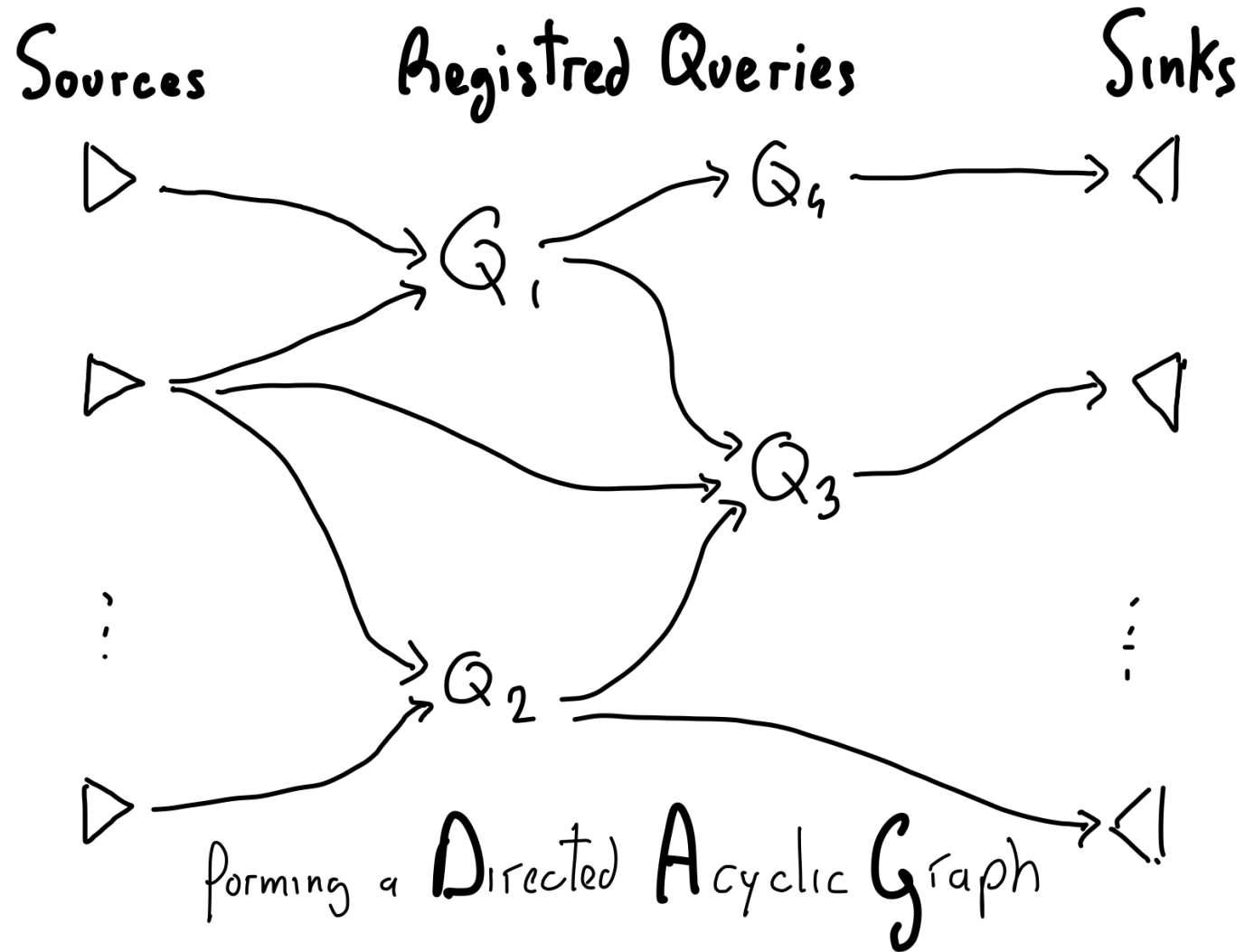
SQL Extensions , e.g. EPL by EsperTech

Stream to stream join

Join the View and Click streams over a 9-second window on id and return all matching records

```
select *  
from View.win:time(9 sec) as v  
    inner join  
    Click.win:time(9 sec) as c  
    on v.id = c.id;
```


Typically queries are placed in a network





Typically queries are placed in a network

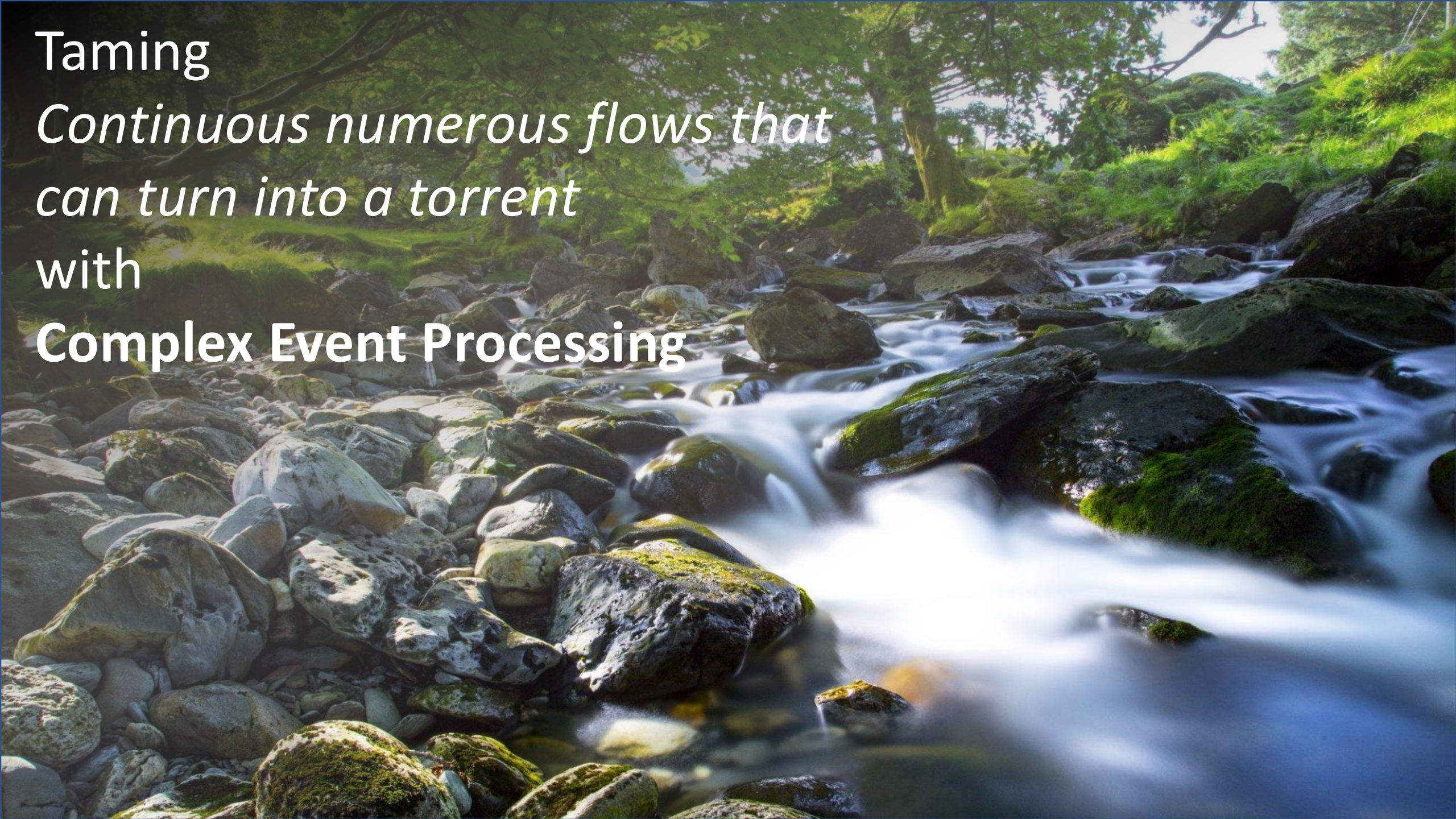


[src: <https://www.confluent.io/blog/confluent-cloud-ksql-as-a-service/>]

Taming

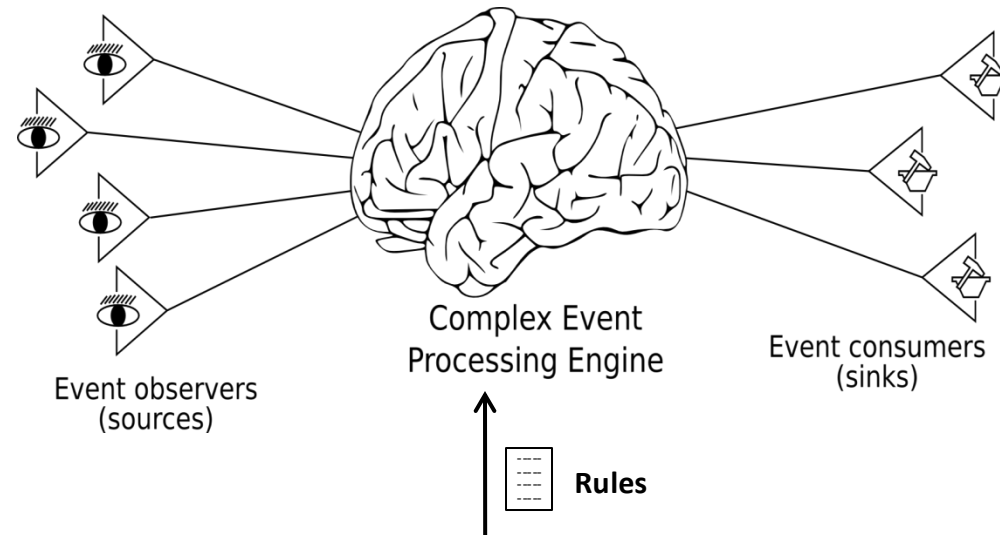
*Continuous numerous flows that
can turn into a torrent
with*

Complex Event Processing



Complex Event Processing (CEP)

- CEP systems adds the ability to deploy *rules* that describe how composite events can be generated from primitive (or composite) ones
- Typical CEP rules search for *sequences of events*
 - Raise C if $A \rightarrow B$
- Time is a key aspect in CEP



Cugola, G., & Margara, A. (2012). **Processing flows of information: From data stream to complex event processing.** *ACM Computing Surveys (CSUR)*, 44(3), 1-62.

SQL Extensions , e.g. EPL by EsperTech

EPL Pattern

```
insert into alertIBM
select *
from pattern [
    every (
        StockTick (name="IBM", price>100)
        ->
        (StockTick (name="IBM", price<100)
            where timer:within(60 seconds))
    ];
```

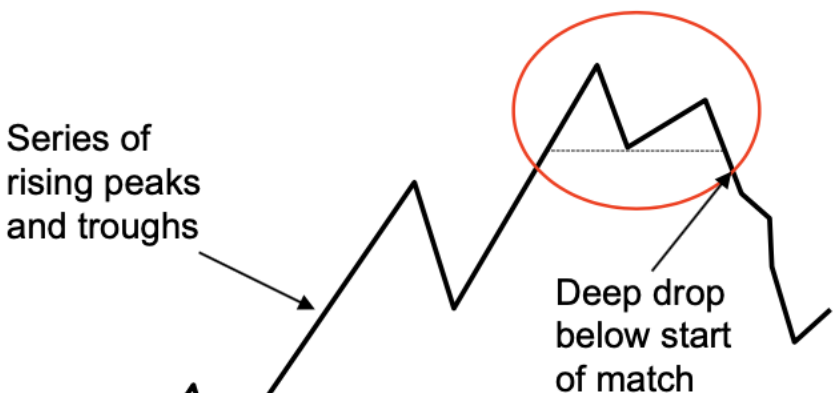
Alert on each IBM stock tick with a price greater than 100, followed by an IBM stock tick lower than 100 within 60 seconds

- Supported since 2006 Esper by EsperTech Inc. and Oracle CEP

Isn't this a join?

- Yes
 - The logical “followed by ->” operator is equivalent to filtering the results of a join
- No
 - Since the logical “followed by ->” operator asks for a recent subset of a join, there are highly efficient physical operators for implementing it
- More to discuss in future lectures

Regular Expressions for CEP



```

1 stream<MatchT> Matches = MatchRegex(Quotes) {
2   param
3   pattern  : ", rise+ drop+ rise+ drop* deep";
4   partitionBy : symbol;
5   predicates : {
6     rise = price>First(price) && price=>Last(price),
7     drop = price=>First(price) && price<Last(price),
8     deep = price<First(price) && price<Last(price) };
9   output
10  Matches : symbol=symbol, seqNum=First(seqNum),
11           count=Count(), maxPrice=Max(price);
12 }
```

Composite events

Simple events

Regular

expression

Key

Aggregation

→ Operator only, no extensions to SPL syntax

SQL Extensions , e.g. EPL by EsperTech

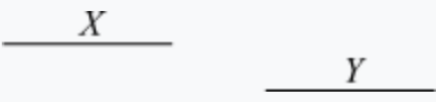
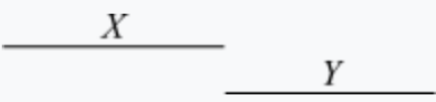
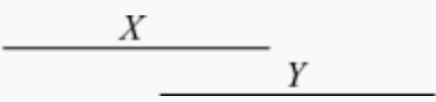
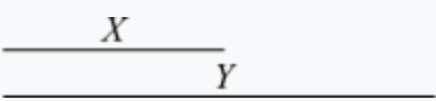
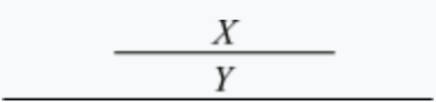
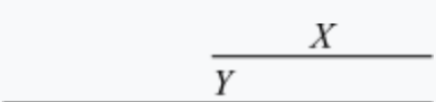
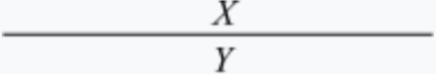
Match Recognize in SQL

look for a sequence of events in which the temperature starts out below 50 and then stays between 50 and 60 and finally comes over 60

```
select * from TemperatureSensorEvent
match_recognize (
  partition by device
  measures A.id as a_id, count(B.id) as count_b, C.id as c_id
  pattern (A B* C)
  define
    A as A.temp < 50,
    B as B.temp between 50 and 60,
    C as C.temp > 60)
```

Are these intervals?

- Somehow
- Patterns allow the implementation of a subset of Allen's Interval Algebra

Relation	Illustration	Interpretation
$X < Y$ $Y > X$		X takes place <u>before</u> Y
$X \text{ m } Y$ $Y \text{ mi } X$		X meets Y (<i>i</i> stands for <i>inverse</i>)
$X \text{ o } Y$ $Y \text{ oi } X$		X overlaps with Y
$X \text{ s } Y$ $Y \text{ si } X$		X starts Y
$X \text{ d } Y$ $Y \text{ di } X$		X during Y
$X \text{ f } Y$ $Y \text{ fi } X$		X finishes Y
$X = Y$		X is equal to Y

[src: https://en.wikipedia.org/wiki/Allen%27s_interval_algebra]

X followed by Y



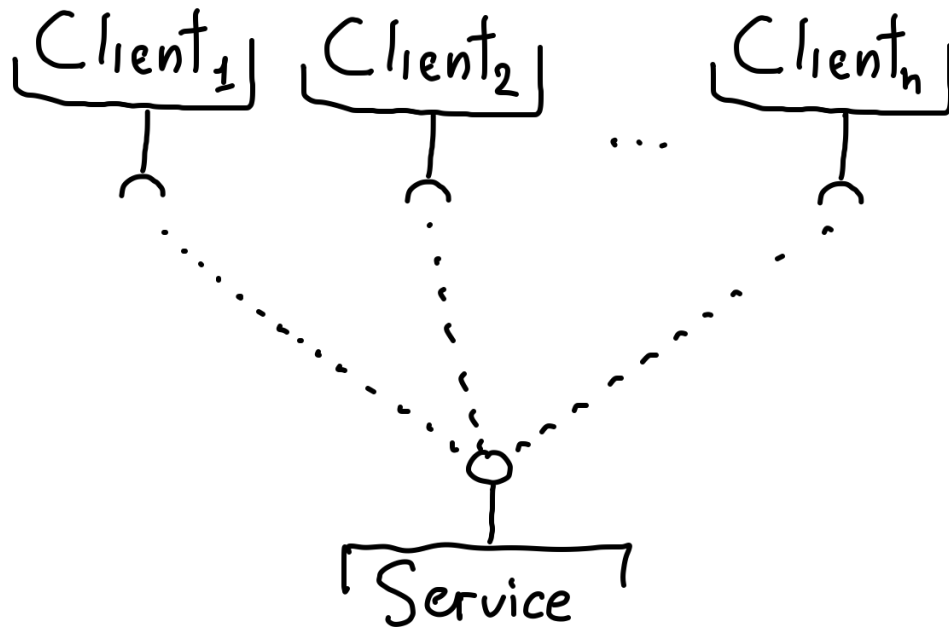
Taming

*Myriads of flows of any size and speed
forming an immense delta
with*

Event-driven Architecture

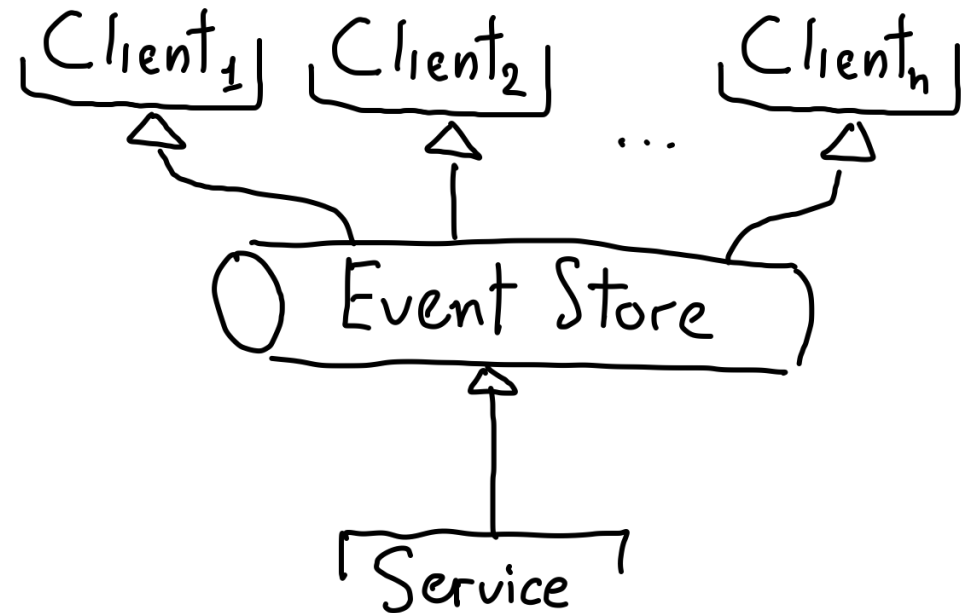
From tight to loose temporal coupling

Service Oriented Architecture



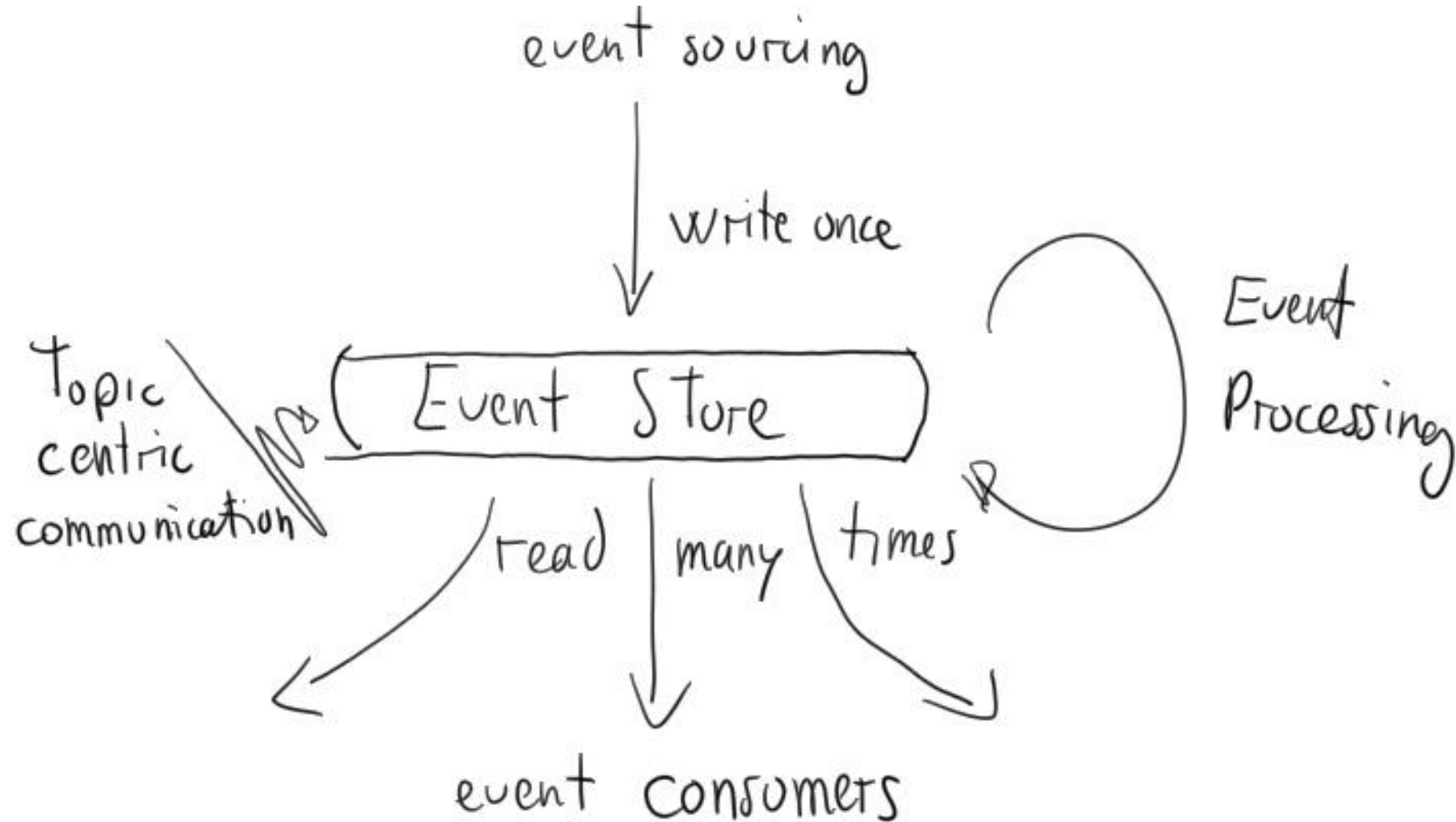
Must answer each request timely

Event-Driven Architecture

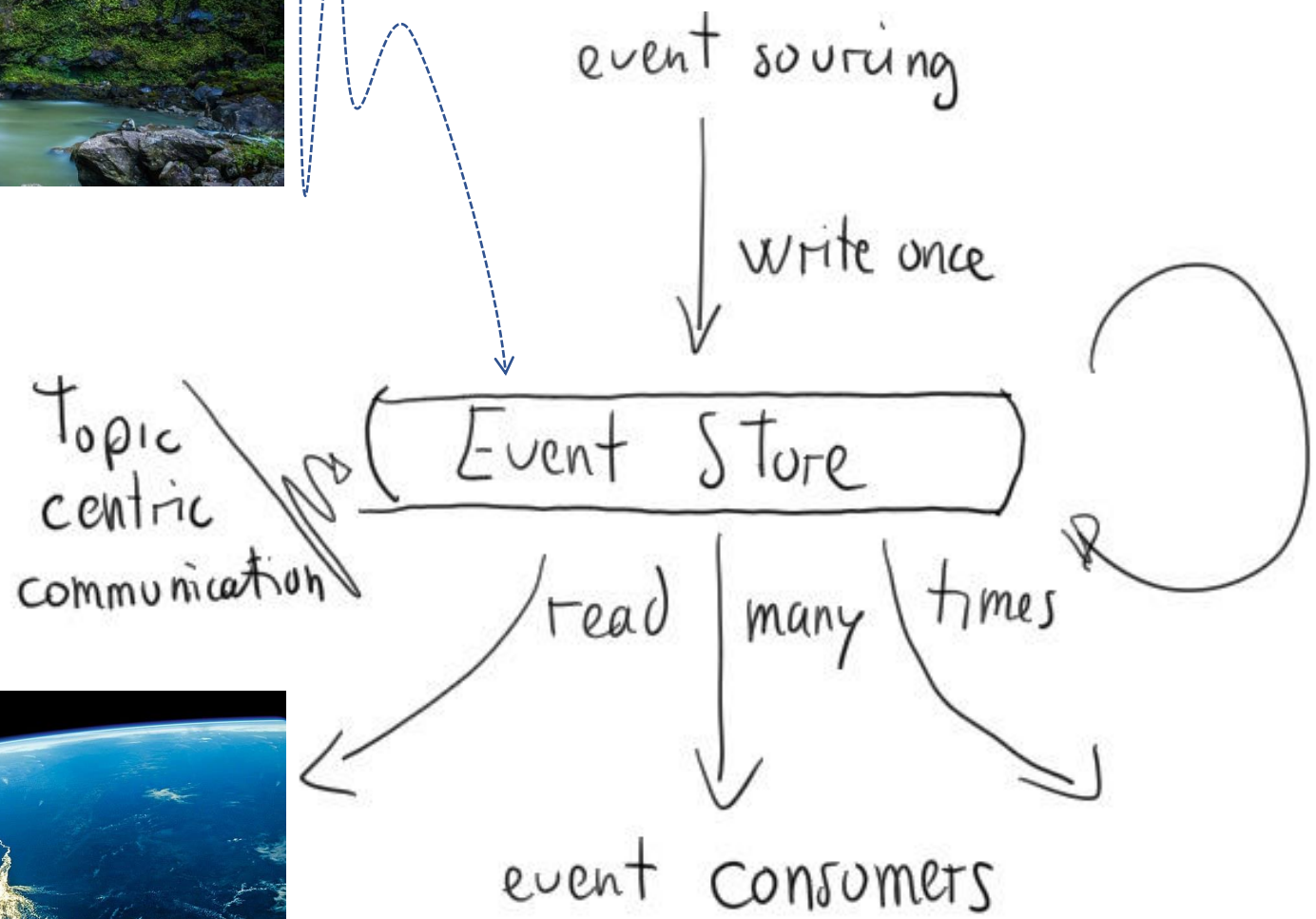


Writes once. Clients read many times asynchronously

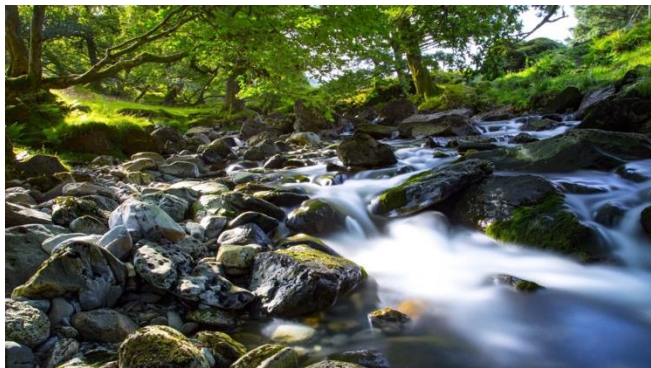
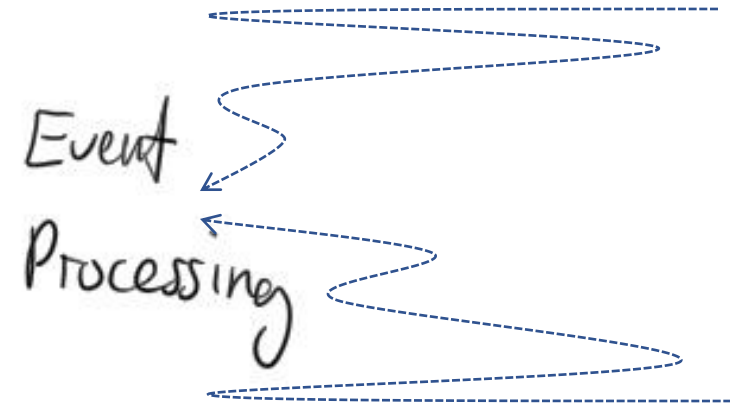
Event processing in Event-driven Architecture



Event-based Systems



Data Stream Mng. Sys.



Complex Event Proc.

Event-driven architecture @ Netflix



[src. <https://labs.sogeti.com/microservices-architectures-stay/>]

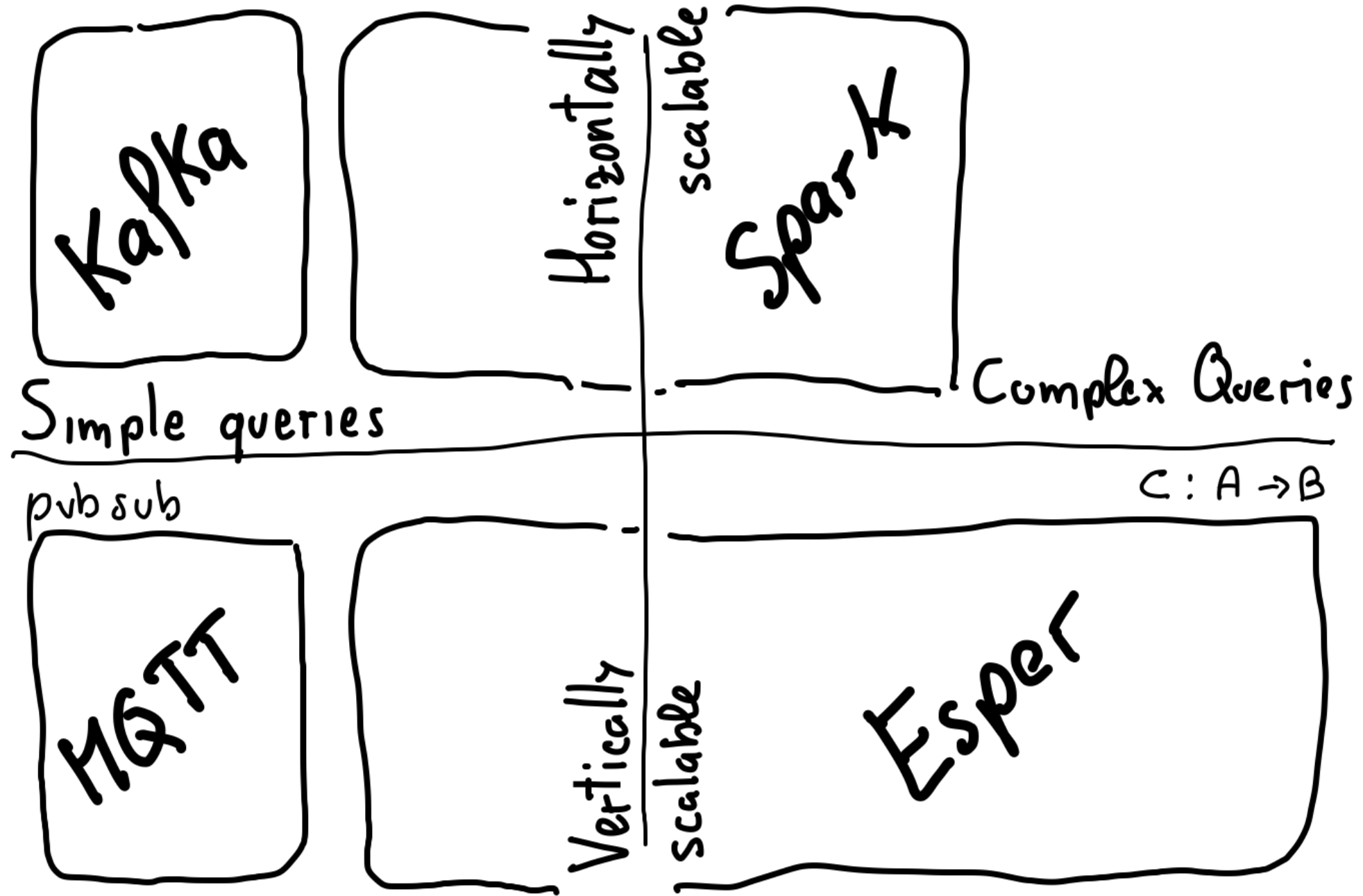


POLITECNICO
MILANO 1863

Wrapping up

Time-models vs. System Types

	Stream only	Absolute time	Interval-based
EBS	X		
DSMS	X	X	
CEP	X	X	~
EDA	X	As a data payload	As a data payload





POLITECNICO
MILANO 1863

Thank you for your attention!

Questions?

Credits

- These slides are partially based on
 - "A modeling framework for DSMS and CEP" by G. Cugola and A. Margara presented in the PhD course on "Stream and complex event processing" offered by Politecnico di Milano in 2015.
 - <http://www.streamreasoning.org/courses/scep2015>
 - "Tutorial: Stream Processing Languages" by Martin Hirzel, IBM Research AI. 1 November 2017 Dagstuhl Seminar on Big Stream Processing Systems
 - <https://materials.dagstuhl.de/files/17/17441/17441.MartinHirzel1.Slides.pdf>
 - "The Death and Rebirth of the Event Driven Architecture" by Jay Kreps, Confluent, Kafka Summit 2018
 - <https://www.confluent.io/kafka-summit-london18/keynote-the-death-and-rebirth-of-the-event-driven-architecture>



POLITECNICO
MILANO 1863

Streaming Data Engineering

A Tale of Four Streams

Emanuele Della Valle

prof @ Politecnico di Milano

founder @ Quantia Consulting

Founder & CRO @ motus ml